

Politechnika
Śląska

PRACA MAGISTERSKA

Ocena przydatności narzędzi do generowania kodu na przykładzie skomplikowanego problemu informatycznego

Paweł SKORUPA
Nr albumu: 295741

Kierunek: Informatyka
Specjalność: Internet i Technologie Sieciowe

PROWADZĄCY PRACĘ
Dr inż. Paweł Benecki
KATEDRA Informatyki Stosowanej
Wydział Automatyki, Elektroniki i Informatyki

Gliwice 2025

Tytuł pracy

Ocena przydatności narzędzi do generowania kodu na przykładzie skomplikowanego problemu informatycznego

Streszczenie

Praca magisterska poświęcona jest ocenie przydatności narzędzi do automatycznego generowania kodu opartych na dużych modelach językowych (ang. *Large Language Model, LLM*) w kontekście rozwiązywania złożonych problemów informatycznych. Na potrzeby analizy opracowano środowisko testowe umożliwiające porównanie wybranych rozwiązań według różnych kryteriów, m.in. poprawność działania, efektywność obliczeniowa i czytelność kodu.

Jako przykład problemu informatycznego wybrano problem komiwojażera, z uwagi na jego wysoką złożoność algorytmiczną i liczne możliwe podejścia do implementacji.

Opisane środowisko testowe oraz metodyka badawcza stanowią podstawę dla przeprowadzenia analizy porównawczej i systematycznego testowania możliwości nowoczesnych narzędzi wspomagających programowanie.

Słowa kluczowe

Sztuczna inteligencja, generowanie kodu, problem komiwojażera, analiza porównawcza, LLM

Thesis title

Evaluating the utility of code generation tools using a complex computational problem

Abstract

This master's thesis is dedicated to evaluating the usefulness of code generation tools based on Large Language Models in the context of solving complex computational problems. For the purpose of the analysis, a testing environment was developed to compare selected solutions based on various criteria, including correctness, computational efficiency, and code readability.

The Traveling Salesman Problem was used as an example of a computational challenge due to its high algorithmic complexity and the wide range of possible implementation approaches.

The described testing environment and research methodology form the basis for a comparative analysis and systematic evaluation of the capabilities of modern code generation tools.

Key words

Artificial intelligence, code generation, travelling salesman problem, comparative analysis, LLM

Spis treści

1 Wstęp **1**

2 Analiza tematu **5**

2.1 Wprowadzenie do zagadnienia i sformułowanie problemu 5

2.2 Problem komiwojażera – charakterystyka i znaczenie 6

2.3 Przegląd wybranych metod rozwiązywania TSP 8

2.3.1 Metody dokładne 8

2.3.2 Algorytmy heurystyczne 9

2.3.3 Metaheurystyki 9

2.3.4 Podejścia hybrydowe 10

2.3.5 Podsumowanie 10

2.4 Generatywne modele językowe w programowaniu 10

2.4.1 Architektura i trening 11

2.4.2 Możliwości generatywnych modeli językowych w kontekście kodu źródłowego 13

2.4.3 Zastosowania praktyczne 13

2.4.4 Ocena jakości kodu generowanego przez LLM 13

2.5 Przegląd badań i literatury dotyczącej AI w kontekście generowania kodu . 14

2.6 Osadzenie pracy w kontekście aktualnych badań 17

3 Przedmiot pracy **19**

3.1 Wybór problemu testowego: problem komiwojażera jako benchmark 19

3.1.1 Motywacja wyboru 19

3.1.2 Zalety problemu w kontekście testowania LLM 19

3.1.3 Rola TSP jako reprezentanta złożonych problemów algorytmicznych 20

3.2 Charakterystyka porównywanych narzędzi AI 21

3.2.1 Lista użytych modeli i środowisk 21

3.2.2 Wersje modeli i tryb dostępu 23

3.2.3 Zasady generowania rozwiązań i konstrukcja zapytań 23

3.3 Struktura testów funkcjonalnych 25

3.3.1 Organizacja kodu testowego 25

3.3.2 Przykład instancji testowej (4x4) . . . 26
3.3.3 Testy instancji o zwiększonej złożoności . . . 27
3.3.4 Testy odporności na błędne dane wejściowe . . . 27
3.3.5 Walidacja ścieżki i kosztu . . . 28
3.3.6 Zasady zaliczenia testów . . . 28
3.4 Środowisko testowe i narzędzia . . . 29
3.4.1 Środowisko IDE – IntelliJ IDEA . . . 29
3.4.2 Analiza jakości kodu – SonarQube (SonarCloud) . . . 30
3.4.3 Pomiar czasu i zużycia pamięci . . . 32

4 Badania **33**
4.1 Analiza rozwiązań dla prostego zapytania (ograniczony kontekst) . . . 34
4.1.1 Przegląd implementacji uzyskanych za pomocą poszczególnych narzędzi . . . 34
4.1.2 Wyniki statycznej analizy i metryki czystości kodu . . . 37
4.1.3 Najczęstsze kategorie problemów wykryte w analizie . . . 38
4.1.4 Przykłady i naprawy typowych problemów (z fragmentami kodu) . 38
4.1.5 Podsumowanie rozwiązań dla ograniczonego kontekstu . . . 40
4.2 Analiza rozwiązań dla złożonego zapytania (pełniejszy kontekst) . . . 41
4.2.1 Przegląd wygenerowanych rozwiązań . . . 41
4.3 Wyniki statycznej analizy kodu (złożone zapytanie) . . . 45
4.4 Analiza poprawności wygenerowanych rozwiązań . . . 45
4.4.1 Odporność na błędne dane wejściowe (złożone zapytanie) . . . 48
4.4.2 Czas wykonania i zajętość pamięci . . . 49

5 Podsumowanie i wnioski **53**
5.1 Sposób rozwiązania problemu . . . 53
5.2 Najważniejsze obserwacje empiryczne . . . 54
5.3 Rekomendacje praktyczne . . . 55
5.3.1 Dokładne zdefiniowanie celu oraz metody rozwiązania . . . 55
5.3.2 Walidacja wejścia/wyjścia - kontrakty poprawności . . . 55
5.3.3 Higiena kodu . . . 55
5.4 Możliwości rozwoju i nowe kierunki . . . 56
5.5 Realizacja celu pracy . . . 56
5.6 Podsumowanie końcowe . . . 57

Bibliografia **59**

Spis skrótów i symboli **65**

Lista dodatkowych plików, uzupełniających tekst pracy **67**

Spis rysunków **69**

Spis tabel **71**

Rozdział 1

Wstęp

W ostatnich latach nastąpił gwałtowny wzrost zainteresowania wykorzystaniem sztucznej inteligencji (ang. *Artificial Intelligence, AI*) w dziedzinie programowania, w szczególności w obszarze automatycznego generowania kodu źródłowego. Dynamiczny rozwój dużych modeli językowych (ang. *Large Language Models, LLM*), takich jak GPT, Claude (Anthropic), Gemini (Google), DeepSeek, czy narzędzi wspomagających programistów takich jak GitHub Copilot czy Tabnine, sprawił, że generatywna sztuczna inteligencja zaczęła być postrzegana jako potencjalnie rewolucyjne narzędzie w pracy inżyniera oprogramowania. Modele te są w stanie przekształcać zapytania w języku naturalnym w gotowe fragmenty kodu, a ich skuteczność wzrasta wraz z rozmiarem kontekstu wejściowego i jakością opisu problemu, co potwierdzają badania IBM [27]. .

Pomimo ogromnego potencjału technologii LLM, pojawia się zasadnicze pytanie: na ile skuteczne i niezawodne są te modele w generowaniu kodu rozwiązującego złożone problemy algorytmiczne? W niniejszej pracy podjęto próbę odpowiedzi na to pytanie poprzez eksperymentalne porównanie jakości, poprawności i efektywności rozwiązań generowanych przez wybrane narzędzia AI, na przykładzie klasycznego problemu komiwojażera (ang. *Travelling Salesman Problem, TSP*). Problem TSP, będący jednym z najbardziej znanych problemów optymalizacyjnych klasy NP-trudnych, jest powszechnie stosowany jako punkt odniesienia do oceny algorytmów heurystycznych, optymalizacyjnych i metaheurystycznych. Jego dobrze określona struktura oraz możliwość łatwego sprawdzenia poprawności i jakości rozwiązania czynią go bardzo dobrym kandydatem do testowania narzędzi AI generujących kod.

Celem pracy jest zbadanie możliwości generatywnej sztucznej inteligencji w kontekście rozwiązywania złożonych problemów obliczeniowych. Przeprowadzono szeroko zakrojone eksperymenty porównawcze z udziałem siedmiu wybranych narzędzi AI: GitHub Copilot, Microsoft Copilot, ChatGPT (OpenAI), Claude (Anthropic), Gemini (Google), DeepSeek oraz Tabnine. Analizowano zarówno poprawność wygenerowanego kodu, jak i jakość struktury programu, jego zgodność ze standardami inżynierii oprogramowania, a także wydajność obliczeniową (czas wykonania, zużycie pamięci). Porównania dokonano

w dwóch scenariuszach: przy pełnym opisie problemu (złożony kontekst wejściowy) oraz w wariancie ograniczonym (minimalny opis funkcjonalny).

Zakres pracy

Zakres pracy obejmuje:

- wybór i przygotowanie zbioru reprezentatywnych narzędzi AI do generowania kodu,
- opracowanie wspólnego środowiska testowego,
- przygotowanie dwóch wersji zapytań testowych: szczegółowego (złożonego) oraz skróconego (minimalnego),
- analizę poprawności funkcjonalnej wygenerowanych programów (czy rozwiązują problem zgodnie z wymaganiami TSP),
- analizę jakości kodu przy pomocy narzędzi takich jak IntelliJ IDEA i SonarQube, które dostarczają obiektywnych metryk czystości kodu (np. ostrzeżenia, błędy logiczne, naruszenia dobrych praktyk),
- pomiar wydajności wygenerowanych programów dla trzech zestawów danych (grafy 4x4, 10x10, 15x15),
- identyfikację błędów wykonania (np. wyjątki, przepełnienie stosu), oraz analizę ich przyczyn.

Struktura pracy

W ramach pracy poszczególne elementy zostały pogrupowane w czterech głównych częściach:

- Analiza tematu (rozdział 2): rozdział ten przedstawia teoretyczne podstawy problemu komiwojażera, jego znaczenie w informatyce teoretycznej i praktyce inżynierskiej, a także przegląd znanych podejść algorytmicznych do jego rozwiązania, takich jak algorytmy przeszukiwania zupełnego (ang. *brute force*), zachłanne (np. algorytm najbliższego sąsiada), dynamiczne, ewolucyjne oraz metaheurystyki. Dodatkowo zaprezentowano aktualny stan wiedzy w dziedzinie generatywnej AI oraz dokonano przeglądu literatury dotyczącej modeli językowych wykorzystywanych do generowania kodu. Opisano architekturę i założenia działania wybranych narzędzi (Copilot, ChatGPT, Claude itd.), wraz z ich miejscem w kontekście obecnych trendów rozwoju AI.

- Przedmiot pracy (rozdział 3): w tej części przedstawiono szczegółowo koncepcję eksperymentu, metodykę porównania oraz motywacje stojące za wyborem problemu TSP jako wyznacznika. Opisano zaprojektowane przez autora rozwiązanie, strukturę testów oraz sposób oceny poprawności działania i jakości wygenerowanego kodu. Przedstawiono też uzasadnienie przyjętych kryteriów oceny (poprawność, jakość kodu, czas i pamięć) oraz sposoby ich pomiaru.

- Badania (rozdział 4): rozdział ten zawiera szczegółową analizę uzyskanych w eksperymencie implementacji algorytmicznych. Analiza rozdzielona jest na dwa przypadki testowe: proste i złożone zapytanie, obejmuje analizę higieny kodu, poprawności rozwiązań oraz ich wydajności czasowej i pamięciowej. Wyniki przedstawiono w postaci tabel i wykresów, a także omówiono błędy występujące w wygenerowanych rozwiązaniach (np. błędy logiczne, błędy wykonania, przepełnienia stosu).

- Podsumowanie i wnioski (rozdział 5): w ostatnim rozdziale syntetycznie podsumowano uzyskane rezultaty, oceniono skuteczność poszczególnych narzędzi. Sformułowano wnioski końcowe, oceniono realizację celu pracy oraz zaproponowano potencjalne kierunki dalszych badań – m.in. testowanie bardziej złożonych problemów, analizę wpływu inżynierii zapytań (ang. *prompt engineering*), czy rozszerzenie porównania o kolejne narzędzia.

Rozdział 2

Analiza tematu

2.1 Wprowadzenie do zagadnienia i sformułowanie problemu

Rozwój generatywnej sztucznej inteligencji na przestrzeni ostatnich kilku lat radykalnie zmienił krajobraz inżynierii oprogramowania. Pojawienie się dużych modeli językowych oraz narzędzi takich jak GitHub Copilot, ChatGPT, Claude, Gemini czy Tabnine wprowadziło do użytku nową kategorię asystentów programistycznych, których głównym zadaniem jest wspomaganie tworzenia kodu źródłowego – zarówno w postaci autouzupełniania, jak i generowania całych bloków programów na podstawie opisu problemu.

Wzrost popularności tych narzędzi wynika z oferowanych przez nie zdolności do skracania czasu implementacji, automatyzacji powtarzalnych zadań, a także obniżenia bariery wejścia w programowanie dla mniej doświadczonych użytkowników. Jednocześnie jednak pojawia się pytanie o ich faktyczną przydatność:

- W jakim zakresie modele te są w stanie generować kod nie tylko syntaktycznie poprawny, lecz także semantycznie zgodny z założeniami użytkownika?
- Czy są w stanie radzić sobie z bardziej złożonymi problemami algorytmicznymi?
- Jaką jakość, wydajność i zgodność z dobrymi praktykami reprezentują generowane rozwiązania?

Celem niniejszej pracy jest przeprowadzenie empirycznej analizy użyteczności i jakości rozwiązań generowanych przez wybrane modele językowe w kontekście rozwiązywania problemów algorytmicznych. W szczególności badanie skupia się na:

- porównaniu skuteczności różnych narzędzi AI w kontekście generowania kodu źródłowego dla określonego problemu (przykładowo problemu komiwojażera),

- ocenie jakości wygenerowanych rozwiązań z punktu widzenia: poprawności działania, efektywności czasowej i pamięciowej, odporności na przypadki brzegowe lub nietypowe dane wejściowe, zgodności z dobrymi praktykami programistycznymi (m.in. na podstawie analiz SonarQube i IntelliJ IDEA),

Problem badawczy można więc sformułować następująco:

Jaką użyteczność i jakość reprezentują współczesne narzędzia AI do generowania kodu, w kontekście ich praktycznego zastosowania do implementacji rozwiązań algorytmicznych?

W celu umożliwienia rzetelnego porównania badane modele zostały poddane testom polegającym na wygenerowaniu kodu rozwiązującego klasyczny problem algorytmiczny – problem komiwojażera. Problem ten, znany ze swojej złożoności obliczeniowej (klasa NP-trudnych), stanowi dobre narzędzie do oceny, czy dane narzędzie AI jest w stanie wygenerować nie tylko działający kod, ale również efektywny, zoptymalizowany i odporny na błędy program.

W kolejnych podrozdziałach omówione zostaną szczegółowo zarówno aspekty związane z samym problemem komiwojażera, jak i działaniem modeli językowych oraz przeglądem literatury poświęconej temu zagadnieniu.

2.2 Problem komiwojażera – charakterystyka i znaczenie

Jednym z klasycznych problemów optymalizacyjnych, szeroko wykorzystywanych w badaniach z zakresu algorytmiki, sztucznej inteligencji oraz inżynierii oprogramowania, jest problem komiwojażera. W niniejszej pracy pełni on rolę przykładu wykorzystywanego do analizy możliwości generatywnych modeli językowych. Jego wybór nie jest przypadkowy – TSP stanowi wymagające zadanie, które łączy w sobie aspekty kombinatoryczne, algorytmiczne oraz implementacyjne, co czyni go użytecznym benchmarkiem (punktem odniesienia) dla oceny jakości wygenerowanego kodu.

Definicja problemu

Problem komiwojażera zdefiniować można następująco:

Definicja 1. *Znajdź najkrótszą trasę dla komiwojażera rozpoczynającą się w danym mieście, odwiedzającą każde z określonej grupy miast, a następnie powracającą do pierwotnego punktu wyruszenia [8].*

Często można się również spotkać z definicją opartą na pojęciu grafu Hamiltona:

Definicja 2. *Cykl Hamiltona to taki cykl w grafie, w którym każdy z wierzchołków odwiedzony jest dokładnie jeden raz. Rozwiązaniem problemu komiwojażera jest cykl Hamiltona o najkrótszej ścieżce.*

Z formalnego punktu widzenia problem zakłada, że dana jest skończona liczba miast $V = \{v_1, v_2, \ldots, v_n\}$ oraz funkcja kosztu $d(v_i, v_j)$, reprezentująca odległość lub koszt przejścia pomiędzy miastami v_i i v_j. Celem jest znalezienie permutacji π miast, która minimalizuje łączny koszt podróży, rozpoczynającej się i kończącej w tym samym mieście, przy założeniu, że każde miasto odwiedzane jest dokładnie raz.

W notacji matematycznej problem ten można zapisać jako:

$$\min_{\pi} \sum_{i=1}^{n-1} d(\pi(i), \pi(i+1)) + d(\pi(n), \pi(1)) \tag{2.1}$$

Dla problemu z n miastami, liczba możliwych tras (permutacji cyklicznych, z pominięciem kierunku) wynosi:

$$\frac{(n-1)!}{2} \tag{2.2}$$

co oznacza, że liczba możliwych rozwiązań rośnie w tempie rzędu silni wraz z liczbą miast – dla 10 miast jest to już $\frac{9!}{2} = 181{,}440$ tras.

Wyróżnia się dwa główne warianty problemu:

- **Symetryczny TSP (STSP)** – koszt przejścia między miastami jest taki sam w obu kierunkach:

$$d(v_i, v_j) = d(v_j, v_i) \tag{2.3}$$

- **Asymetryczny TSP (ATSP)** – koszt może się różnić w zależności od kierunku:

$$d(v_i, v_j) \neq d(v_j, v_i) \tag{2.4}$$

TSP należy do klasy problemów NP-trudnych, co oznacza, że brak jest znanego algorytmu rozwiązującego go w czasie wielomianowym dla dowolnej liczby miast. Z tego względu jest to jeden z najlepiej przebadanych problemów optymalizacyjnych, a zarazem klasyczny przypadek testowy dla nowych podejść algorytmicznych – zarówno deterministycznych (np. programowanie dynamiczne, metody dokładne), jak i przybliżonych (heurystyki, metaheurystyki, algorytmy ewolucyjne itp.).

Zastosowania praktyczne

Pomimo swojej teoretycznej natury, problem komiwojażera ma liczne praktyczne zastosowania, m.in. w:

- logistyce i transporcie (optymalizacja tras dostaw, planowanie przejazdów),
- planowaniu produkcji (np. w elektronice – optymalizacja ruchu głowicy lutującej),
- biologii obliczeniowej (np. rekonstrukcja genomów),
- robotyce i sterowaniu ruchem (nawigacja autonomiczna),
- kompresji danych i wizualizacji grafów.

Reprezentatywność dla oceny modeli AI

W kontekście niniejszej pracy, wybór TSP pozwala na wielowymiarową ocenę możliwości narzędzi opartych na dużych modelach językowych:

- **Złożoność algorytmiczna:** problem wymaga zastosowania przynajmniej podstawowych technik przeszukiwania (np. brute-force, algorytmy zachłanne, heurystyki).
- **Zdolność do generacji poprawnego kodu:** zadanie to wymusza stosowanie struktur danych, pętli, instrukcji warunkowych, co stanowi test umiejętności syntaktycznych i logicznych.

2.3 Przegląd wybranych metod rozwiązywania TSP

Wieloletnie badania nad problemem komiwojażera doprowadziły do powstania różnych klas metod jego rozwiązywania. Należą do nich zarówno techniki dokładne, jak i szeroki zestaw podejść heurystycznych oraz metaheurystycznych, stosowanych zwłaszcza dla dużych i trudnych przypadków. Poniżej przedstawiono najważniejsze z metod rozwiązywania tego problemu.

2.3.1 Metody dokładne

Metody te gwarantują znalezienie optymalnego rozwiązania, jednak ich koszt obliczeniowy jest zbyt wysoki dla dużych instancji (np. powyżej 20–30 wierzchołków). Do najważniejszych należą:

- **Przeszukiwanie zupełne (*brute-force*)** – polega na sprawdzeniu wszystkich możliwych permutacji tras. Liczba możliwych tras wynosi $\frac{(n-1)!}{2}$ w przypadku problemu symetrycznego oraz $(n-1)!$ w przypadku problemu asymetrycznego, co sprawia, że metoda pełnego przeszukiwania staje się niepraktyczna dla większych wartości n. Choć koncepcyjnie prosta, metoda ta służy raczej jako punkt odniesienia przy testowaniu jakości innych algorytmów.

- **Programowanie dynamiczne** – algorytm Bellmana-Helda-Karpa rozwiązuje TSP w czasie $O(n^2 \cdot 2^n)$, co stanowi istotne przyspieszenie w stosunku do zupełnego przeszukiwania [3]. Działa rekurencyjnie, zapamiętując najkrótsze ścieżki prowadzące do zbiorów miast, co pozwala uniknąć wielokrotnego przeliczania tych samych fragmentów trasy.
- **Metody programowania całkowitoliczbowego (ang. *Integer Linear Programming, ILP*)** – modelowanie TSP jako problemu programowania liniowego z dodatkowymi ograniczeniami zapobiegającymi cyklom podrzędnym. Rozwiązywane są przy użyciu metod typu *branch-and-bound* [17]. Kluczowe jest tu dynamiczne dodawanie ograniczeń cięć (*cutting planes*), które eliminują rozwiązania niebędące pojedynczym cyklem Hamiltona.

2.3.2 Algorytmy heurystyczne

Heurystyki nie gwarantują znalezienia optymalnego rozwiązania, ale działają szybko i zwracają rozwiązania przybliżone.

- **Najbliższego sąsiada (*Nearest Neighbor*)** – zaczyna z losowego miasta i w każdej iteracji wybiera najbliższe nieodwiedzone. Prosty, ale często daleki od optimum. Algorytm ten tworzy trasę w sposób zachłanny, ignorując globalną strukturę problemu, co może skutkować pułapkami lokalnych minimów [28].
- **Algorytm dwuspójny (2-opt)** – iteracyjnie poprawia trasę przez zamianę krawędzi w celu eliminacji skrzyżowań [7]. W każdej iteracji porównuje długości tras przed i po wymianie dwóch krawędzi – jeśli nowa trasa jest krótsza, to zmiana zostaje zaakceptowana. Proces ten powtarza się aż do osiągnięcia lokalnego minimum.

2.3.3 Metaheurystyki

Są to strategie inspirowane procesami naturalnymi lub społecznymi. Choć nie gwarantują optymalności, często znajdują bardzo dobre rozwiązania dla dużych instancji.

- **Algorytmy genetyczne (ang. *Genetic Alghorithms, GA*)** – operują na populacji tras, stosując selekcję, krzyżowanie i mutację [13]. Nowe trasy tworzy się przez łączenie fragmentów tras rodzicielskich, a następnie wprowadza niewielkie losowe zmiany, by utrzymać różnorodność w populacji.
- **Symulowane wyżarzanie (ang. *Simulated Annealing*)** – inspirowany zjawiskiem wyżarzania w metalurgii. Opiera się na probabilistycznym przyjmowaniu gorszych rozwiązań w celu uniknięcia lokalnych minimów [16]. Z czasem „temperatura" algorytmu spada, ograniczając możliwość akceptacji gorszych tras i stopniowo koncentrując się na eksploatacji rozwiązań obiecujących.

- **Algorytm mrówkowy (ang. *Ant Colony Optimization*)** – inspirowany zachowaniem kolonii mrówek, które zostawiają feromony na trasach [10]. Im częściej dana trasa jest wybierana przez mrówki, tym większe staje się stężenie feromonów, co zwiększa szansę jej ponownego wyboru, prowadząc do wzmocnienia dobrych rozwiązań.
- **Optymalizacja rojem cząstek (ang. *Particle Swarm Optimization, PSO*)** – modeluje rozwiązywanie problemu przez „cząstki" poruszające się w przestrzeni rozwiązań [6]. Każda cząstka aktualizuje swoje „położenie" (trasę) na podstawie własnego dotychczas najlepszego rozwiązania oraz najlepszego rozwiązania wśród wszystkich cząstek, co sprzyja eksploracji i konwergencji.

2.3.4 Podejścia hybrydowe

W ostatnich latach rozwijane są metody łączące klasyczne podejścia z uczeniem maszynowym. Przykładem mogą być:

- **GNN-TSP** – zastosowanie grafowych sieci neuronowych (ang. *Graph Neural Network*) do bezpośredniego przewidywania tras [15]. Model GNN uczy się reprezentacji wierzchołków i relacji między nimi, co pozwala mu przewidywać prawdopodobieństwo występowania danej krawędzi w rozwiązaniu.
- **Uczenie ze wzmocnieniem dla TSP** – modele uczone w schemacie aktor-krytyk do konstrukcji tras [4]. Sieć decyduje, które miasto odwiedzić w następnej kolejności, a nagroda oparta jest na całkowitej długości trasy.
- **Generowanie heurystyk z użyciem LLM** – jak wskazują autorzy pracy [11], LLM mogą nie tylko generować kod, ale również projektować heurystyki ad hoc na podstawie tekstowego opisu problemu. Takie podejście testuje zdolność modeli językowych do przenoszenia wiedzy algorytmicznej między zadaniami.

2.3.5 Podsumowanie

Wybór odpowiedniego podejścia zależy od rozmiaru zadanego zbioru miast, wymagań czasowych oraz celu (dokładność / szybkość). W niniejszej pracy metody te służą jako punkt odniesienia przy ocenie jakości kodu generowanego przez modele językowe, z naciskiem na ich skuteczność implementacji logiki takich algorytmów.

2.4 Generatywne modele językowe w programowaniu

Rozwój dużych modeli językowych, takich jak GPT, PaLM, LLaMA czy Claude, znacząco wpłynął na obszar automatycznego programowania. Modele te, trenowane na

ogromnych zbiorach danych tekstowych zawierających także kod źródłowy, umożliwiają generowanie poprawnych składniowo i semantycznie fragmentów programów, często na poziomie porównywalnym z doświadczonym programistą.

2.4.1 Architektura i trening

Większość współczesnych modeli generatywnych oparta jest na architekturze transformera [31], która zrewolucjonizowała dziedzinę przetwarzania języka naturalnego i została zaadaptowana do generowania kodu źródłowego. W kontekście kodu, modele te uczą się statystycznych zależności między tokenami (np. słowami kluczowymi, identyfikatorami, strukturami sterującymi) w sposób analogiczny do naturalnego języka.

Architektura transformera

Architektura transformera składa się z dwóch głównych komponentów: kodera i dekodera, chociaż w przypadku modeli generatywnych kodu często wykorzystuje się wyłącznie dekoder. Koder przetwarza sekwencję wejściową (np. zapytanie opisujące zadanie programistyczne) poprzez serię warstw uwagi i przekształceń, tworząc bogatą reprezentację kontekstową, która koduje semantyczne relacje między elementami wejścia. Dekoder natomiast generuje sekwencję wyjściową (kod źródłowy) w sposób autoregresyjny - oznacza to, że każdy kolejny token jest przewidywany na podstawie wszystkich wcześniej wygenerowanych tokenów w sekwencji, przy czym model nie ma dostępu do przyszłych tokenów. Ten proces sekwencyjny naśladuje naturalny sposób pisania kodu przez programistów, którzy konstruują program linia po linii, wykorzystując kontekst wcześniej napisanego kodu do podejmowania decyzji o kolejnych instrukcjach.

Kluczowym mechanizmem transformera jest uwaga wielogłowicowa (ang. *multi-head attention*), która pozwala modelowi na równoczesne analizowanie różnych aspektów relacji między tokenami w sekwencji. Mechanizm uwagi oblicza wagi dla każdej pary tokenów, umożliwiając modelowi skupienie się na najbardziej istotnych fragmentach kodu podczas generowania. W kontekście programowania, pozwala to na modelowanie zależności między zmiennymi, funkcjami czy strukturami sterującymi, nawet gdy są one odległe w sekwencji kodu.

Proces treningu

Proces treningu modeli generatywnych kodu składa się z kilku etapów i wykorzystuje różnorodne techniki optymalizacji. Pierwszy etap, zwany *pre-trainingiem*, polega na uczeniu modelu na ogromnych korpusach kodu źródłowego zebranego z repozytoriów open-source, głównie z platform takich jak GitHub, GitLab czy SourceForge.

Korpus treningowy typowo zawiera kod w dziesiątkach języków programowania, dokumentację, komentarze oraz dodatkowy kontekst, taki jak komunikaty *commit* czy opisy

problemów znajdujące się w repozytorium. Dane te są tokenizowane przy użyciu specjalistycznych tokenizatorów. Tokeny mogą reprezentować pojedyncze znaki, słowa kluczowe, identyfikatory, operatory czy nawet całe konstrukcje składniowe.

Głównym zadaniem treningowym jest samouzupełnianie kodu (ang. *next-token prediction*), gdzie model uczy się przewidywać następny token w sekwencji na podstawie kontekstu poprzedzających tokenów. Funkcja straty mierzy różnicę między przewidywaniami modelu a rzeczywistymi tokenami w korpusie.

Drugi etap treningu, dostrajanie (ang. *fine-tuning*), dostosowuje model do specyficznych zadań programistycznych. Może to obejmować:

- Nadzorowane dostrajanie na sparowanych przykładach zapytanie-kod
- Uczenie ze wzmocnieniem z ludzką informacją zwrotną (ang. *Reinforced Learning from Human Feedback, RLHF*)
- Trening na specjalistycznych zadaniach, takich jak refaktoryzacja czy debugowanie.

Modele takie jak Codex [5], CodeGen [23] czy StarCoder [19] są specjalnie dostrojone do zadań kodowania, wykorzystując wymienione powyżej metody.

Wyzwania i ograniczenia

Proces treningu generatywnych modeli kodu napotyka na specyficzne wyzwania. Jednym z najważniejszych jest zjawisko halucynacji, gdzie model generuje kod składniowo poprawny, ale semantycznie nieprawidłowy lub niezgodny z intencją programisty. Halucynacje mogą objawiać się jako:

- Wywoływanie nieistniejących funkcji czy bibliotek
- Nieprawidłowe wykorzystanie interfejsu projektowania aplikacji (ang. *Application Programming Interface, API*)
- Generowanie kodu, który kompiluje się, ale nie wykonuje zamierzonej funkcjonalności
- Wprowadzanie subtelnych błędów logicznych trudnych do wykrycia.

Zjawisko to wynika z faktu, że modele uczą się wyłącznie ze statystycznych wzorców w danych treningowych, bez prawdziwego zrozumienia semantyki kodu czy możliwości jego wykonania. Dodatkowo, modele mogą reprodukować błędy obecne w korpusie treningowym.

2.4.2 Możliwości generatywnych modeli językowych w kontekście kodu źródłowego

Generatywne modele językowe umożliwiają:

- tworzenie funkcji implementujących opisaną słownie logikę,
- uzupełnianie istniejącego kodu,
- tłumaczenie kodu między językami programowania,
- refaktoryzowanie oraz dokumentowanie kodu,
- odpowiadanie na pytania dotyczące działania fragmentów programu.

2.4.3 Zastosowania praktyczne

LLM znajdują zastosowanie w:

- środowiskach IDE (np. GitHub Copilot, Amazon CodeWhisperer),
- systemach wsparcia technicznego i recenzjach kodu,
- automatyzacji testów jednostkowych,
- generowaniu algorytmów,
- wspomaganiu nauki programowania.

Szczególnie interesujące są przypadki, w których modele generują kod rozwiązujący klasyczne problemy algorytmiczne, takie jak problem komiwojażera, co jest przedmiotem dalszej analizy w niniejszej pracy.

2.4.4 Ocena jakości kodu generowanego przez LLM

W literaturze istnieje wiele prób systematycznej oceny modeli językowych w kontekście programowania [25][29][32]. Najczęściej stosowane metody obejmują:

- testy jednostkowe dla wygenerowanych funkcji,
- porównania z rozwiązaniami referencyjnymi,
- analizę syntaktyczną (np. poprawność kompilacji),
- ocenę stylu i zgodności z praktykami inżynierii oprogramowania.

W przypadku zadań algorytmicznych dodatkowym kryterium jest poprawność logiczna implementacji, którą trudno ocenić bez uruchomienia kodu. Z tego powodu popularne są benchmarki takie jak HumanEval [5] czy MBPP [2], gdzie każde zadanie zawiera zestaw testów wejściowych i oczekiwanych wyjść.

2.5 Przegląd badań i literatury dotyczącej AI w kontekście generowania kodu

Benchmarking ChatGPT, Codeium i GitHub Copilot Artykuł [25] prezentuje porównanie trzech narzędzi – ChatGPT, Codeium i Copilot – w kontekście 300 zadań z platformy LeetCode [18] (100 łatwych, 100 średnich, 100 trudnych). Analiza obejmuje poprawność rozwiązań, czas wykonania, zużycie pamięci oraz efektywność w debugowaniu.

Autorzy zaobserwowali, że:

- GitHub Copilot dominował w zadaniach łatwych i średnich, osiągając najwyższy odsetek przyjętych rozwiązań.
- ChatGPT wyróżniał się efektywnością pamięciową oraz zdolnością do samodzielnego debugowania — w fazie poprawiania kodu uzyskał wyższy odsetek skorygowanych błędów w porównaniu do Copilota i Codeium.
- Codeium, choć skuteczny w prostych przypadkach, wykazywał znacznie słabsze wyniki w zadaniach trudnych i debugowaniu.

Wnioski te potwierdzają, że choć narzędzia oparte na LLM dobrze radzą sobie w standardowych zadaniach, ich wydajność spada przy problemach bardziej złożonych, co wpisuje się w tematykę niniejszej pracy i wskazuje lukę — szczególnie gdy porównujemy efektywność generowanego kodu z uwzględnieniem bardziej zaawansowanych metryk.

Charakterystyka kodu generowanego przez LLM na GitHubie W pracy [20] autorzy przeanalizowali, jak często i w jakim kontekście na GitHubie wykorzystywane są narzędzia LLM – przede wszystkim ChatGPT i GitHub Copilot. Badanie objęło tysiące repozytoriów zawierających fragmenty wygenerowane przez AI, przy czym kluczowe wnioski są następujące:

- *Przewaga ChatGPT/Copilota* – stanowią najczęściej używane narzędzia LLM na GitHubie, podczas gdy inne modele są praktycznie nieobecne.
- *Typowe repozytoria* – niewielkie projekty prowadzone przez pojedyncze osoby lub małe zespoły, które mimo to wykazują aktywne rozwijanie kodu.
- *Języki i zastosowania* – Python, Java, TypeScript dominują w skryptach do przetwarzania danych; C i C++ najczęściej używane są do implementacji struktur danych, natomiast do tworzenia interfejsów głównie wykorzystywany jest Javascript.
- *Prostota i stabilność kodu* – fragmenty AI wykazują niewielką złożoność (zazwyczaj 12–36 linii), a późniejsze poprawki usuwające błędy stanowią zaledwie 3–8% zmian.

- *Brak metadanych w komentarzach* – większość komentarzy informuje jedynie, że kod jest „AI-generated", bez podawania promptów, testów czy szczegółów modyfikacji.

To podejście dostarcza empirycznych danych na temat realnego wykorzystania LLM-ów w programowaniu — w odróżnieniu od syntetycznych benchmarków — i potwierdza, że generowany kod jest prosty, mało zmieniany, a jego integracja w projektach nadal ograniczona.

Ocena ChatGPT, Gemini, AlphaCode i GitHub Copilot W artykule [29] przeprowadzono porównanie czterech asystentów programistycznych opartych na AI — Chat GPT, Gemini (Bard AI), AlphaCode i GitHub Copilot — wykorzystując zadania z zakresu przetwarzania języka naturalnego (ang. *Natural Language Proccessing, NLP*) oraz generowanie kodu w językach Java, Python i C++. Testy obejmowały zarówno różne poziomy trudności, jak i funkcje debugowania i refaktoryzacji.

Autorzy podkreślają, że:

- **ChatGPT** i **Github Copilot** osiągają dobre wyniki w generowaniu kodu, ale różnią się budową wygenerowanego kodu i strukturą logiki.
- **AlphaCode** lepiej radzi sobie z zadaniami konkurencyjnymi, ale generowany kod bywa bardziej złożony i trudniejszy do debugowania.
- **Gemini** wykazuje najmniejszą stabilność — szczególnie przy zmianach kontekstu — co wskazuje na potrzebę dalszej optymalizacji adaptacji modelu.

Autorzy zauważają także, że choć wszystkie cztery narzędzia poprawiają produktywność programistów i ułatwiają debugowanie, to mają ograniczenia w kontekście czytelności kodu i spójności między sesjami, co jest spójne z wnioskami innych badań. Artykuł porusza również kwestię etycznych aspektów powszechnego stosowania sztucznej inteligencji. W szczególności zwraca uwagę na niepewny status prawny fragmentów kodu generowanych w oparciu o dane treningowe (kwestie własności intelektualnej), potencjalne skutki automatyzacji dla rynku pracy programistów oraz rosnący ślad środowiskowy związany z trenowaniem i wykorzystywaniem dużych modeli językowych.

Jakość kodu generowanego przez Copilot, CodeWhisperer i ChatGPT W badaniu [32] przeanalizowano jakość kodu generowanego przez GitHub Copilot, Amazon CodeWhisperer i ChatGPT przy użyciu benchmarku HumanEval. Autorzy oceniali pięć kluczowych metryk: poprawność składniową (ang. *Validity*), poprawność działania (ang. *Correctness*), bezpieczeństwo (ang. *Security*), niezawodność (ang, *Reliability*) i łatwość utrzymania (ang. *Maintainability*).

Wyniki (wariant „najlepszy z wielu prób") pokazują:

- **Poprawność składniowa**: wszystkie trzy narzędzia osiągnęły wysoki wynik (około 90%), co oznacza, że niemal każdy wygenerowany program dawał się uruchomić bez błędów składniowych.
- **Poprawność działania**: ChatGPT – 65,2%, Copilot – 46,3%, CodeWhisperer – 31,1%.
- **Bezpieczeństwo (Security)** i **Niezawodność (Reliability)**: porównywalne między narzędziami – większość wygenerowanego kodu nie zawierała poważnych błędów ani luk bezpieczeństwa.
- **Łatwość utrzymania (Maintainability)**: wszystkie narzędzia generowały kod z zauważalną liczbą „code smells", przy czym różnice nie były duże. Średni czas usuwania „code smells" dla poszczególnych narzędzi wynosił: ChatGPT – 8,9 min, Copilot – 9,1 min, CodeWhisperer – 5,6 min.

Autorzy zauważają, że choć Copilot i CodeWhisperer zrobiły postępy w kolejnych wersjach (skorygowane o +18 p.p. i +7 p.p. poprawnych rozwiązań), to nadal znacząco odstają pod względem jakości i konserwacji kodu w porównaniu z ChatGPT. Wyniki potwierdzają, że narzędzia LLM mogą generować funkcjonalny, ale często słabo utrzymany kod, co podkreśla potrzebę krytycznej analizy generowanych rozwiązań.

Wpływ Copilota na realne projekty Autorzy [26] badali wpływ GitHub Copilot na 15 typowych zadań inżynierskich wykonywanych przez 26 deweloperów w środowisku „cloud-first" w firmie Cisco. Analiza dotyczyła czasu potrzebnego na dokumentację, pisanie kodu, generowanie testów jednostkowych oraz debugowanie.

Z ich wyników wynika:

- **Oszczędności czasowe**: do 50% przy dokumentacji i autouzupełnianiu, oraz od 30 do 40% przy zadaniach powtarzalnych, testach i debugowaniu.
- **Spadek efektywności** w przypadku zadań o wysokiej złożoności (wielofunkcyjne, wiele plików), szczególnie w C/C++ – w takich kontekstach Copilot uzyskał tylko 26% oszczędności czasu.
- **Różnice między językami**: najlepsze wyniki w Pythonie, Go i JavaScript; niższe w C/C++.
- **Wyzwania: jakość kodu**, kwestie bezpieczeństwa i doświadczenie deweloperów – generowany kod wymagał uważnej weryfikacji.

Artykuł dostarcza empirycznych dowodów na konkretny wpływ Copilota na produktywność, jednocześnie wskazując ograniczenia w realnych projektach — co ściśle wpisuje się w zakres niniejszej pracy, która dąży do oceny użyteczności LLM w kontekście algorytmicznej złożoności i oceny jakości wygenerowanego kodu.

2.6 Osadzenie pracy w kontekście aktualnych badań

W ostatnich latach obserwujemy dynamiczny rozwój badań nad zastosowaniem generatywnych modeli językowych w kontekście automatycznego generowania kodu. Prace te koncentrują się zarówno na aspektach jakościowych (czytelność, poprawność i bezpieczeństwo kodu), jak i ilościowych (czas wykonania, efektywność pamięciowa, wskaźnik sukcesu w testach). W ramach przeglądu literatury można wyróżnić dwa główne nurty badawcze: ocena skuteczności narzędzi LLM w syntetycznych benchmarkach (np. HumanEval, LeetCode) oraz analiza ich wpływu w praktycznych, rzeczywistych środowiskach programistycznych (np. GitHub, środowiska zespołów developerskich).

W odniesieniu do syntetycznych testów, autorzy takich prac jak [25] czy [32] pokazują, że choć narzędzia takie jak ChatGPT czy GitHub Copilot osiągają wysoką skuteczność w zadaniach niskiej i średniej trudności, ich wyniki drastycznie spadają przy bardziej złożonych przypadkach. Jednocześnie wyniki te są zwykle analizowane z perspektywy statystycznej poprawności (czy program działa, czy testy przechodzą), rzadziej zaś z punktu widzenia szczegółowych aspektów implementacyjnych — takich jak struktura kodu, czytelność, zgodność ze wzorcami projektowymi czy efektywność algorytmiczna.

Z kolei prace takie jak [20] oraz [26] skupiają się na empirycznej analizie wykorzystania LLM w środowiskach rzeczywistych – np. w projektach typu open source czy zespołach inżynierskich. Podkreślają one, że choć narzędzia generatywne zwiększają produktywność i ułatwiają powtarzalne zadania, to ich zastosowanie w bardziej złożonych kontekstach pozostaje ograniczone. Problemy związane z utrzymaniem kodu, jego refaktoryzacją czy integracją z istniejącym systemem nadal stanowią istotne bariery.

Na tle powyższych badań, niniejsza praca magisterska wpisuje się w aktualny nurt badawczy, ale jednocześnie uzupełnia go o istotny element: szczegółową analizę jakościową i techniczną kodu wygenerowanego przez różne narzędzia AI, na przykładzie jednego wyraźnie zdefiniowanego i złożonego problemu algorytmicznego — problemu komiwojażera. W odróżnieniu od dotychczasowych podejść skupiających się na wskaźnikach sukcesu lub syntaktycznej poprawności, celem tej pracy jest zbadanie:

- jak różne modele LLM implementują konkretne algorytmy (np. brute-force, heurystyki),
- w jakim stopniu generowany kod jest poprawny, zrozumiały, efektywny i zgodny ze specyfikacją,
- oraz jak różny kontekst wejściowy (prompt) wpływa na jakość generowanych rozwiązań.

Podejście to pozwala z jednej strony zbliżyć się do praktycznych aspektów programowania z użyciem LLM, a z drugiej – ocenić te narzędzia nie tylko jako „czarne skrzynki”

generujące kod, lecz także jako systemy zdolne (lub niezdolne) do reprezentowania i implementowania złożonej logiki algorytmicznej.

W ten sposób praca ta wypełnia lukę między syntetycznymi benchmarkami a realnymi przypadkami użycia, dostarczając pogłębionej oceny użyteczności narzędzi AI w kontekście wymagających problemów informatycznych.

Rozdział 3

Przedmiot pracy

3.1 Wybór problemu testowego: problem komiwojażera jako benchmark

Jednym z kluczowych założeń niniejszej pracy było dobranie takiego problemu testowego, który umożliwiłby możliwie wieloaspektową ocenę jakości kodu generowanego przez narzędzia oparte na dużych modelach językowych. W tym celu zdecydowano się na wykorzystanie klasycznego problemu komiwojażera, który dobrze spełnia wymagania stawiane benchmarkowi w kontekście oceny umiejętności algorytmicznego rozumowania.

3.1.1 Motywacja wyboru

Wybór problemu komiwojażera jako głównego przypadku testowego w niniejszej pracy podyktowany był potrzebą znalezienia zadania, które w realistyczny sposób odwzorowuje wyzwania związane z algorytmicznym rozwiązywaniem problemów w programowaniu. TSP jest problemem dobrze poznanym, a jednocześnie na tyle złożonym, że stanowi wymagający przypadek dla narzędzi generujących kod — zarówno pod względem logiki algorytmu, jak i jakości wynikowego programu.

W przeciwieństwie do prostych zadań (np. sortowania czy iteracji po tablicach), TSP wymaga od narzędzia analizy ogólnej struktury problemu optymalizacyjnego, zaprojektowania odpowiedniego przebiegu obliczeń, oraz zapewnienia zgodności rozwiązania ze zdefiniowanym interfejsem. Tym samym stanowi on dobry test rzeczywistych zdolności modeli językowych w zakresie rozwiązywania problemów programistycznych i generowania spójnego, funkcjonalnego kodu.

3.1.2 Zalety problemu w kontekście testowania LLM

TSP spełnia szereg warunków pożądanych dla problemu benchmarkowego w badaniach nad jakością generowanego kodu:

- **Wysoka złożoność obliczeniowa** — jako problem NP-trudny, TSP nie posiada znanego algorytmu działającego w czasie wielomianowym. Rozwiązanie go wymaga zastosowania struktur danych, pętli zagnieżdżonych, przemyślanej organizacji danych i logiki kontrolnej.

- **Wiele możliwych podejść** — ten sam problem może być rozwiązany na wiele sposobów: od brute-forcowej enumeracji wszystkich możliwych permutacji, przez heurystyki (np. najbliższego sąsiada), aż po metaheurystyki i metody probabilistyczne. Dzięki temu możliwa jest obserwacja, jakie strategie preferują konkretne modele AI i w jakim stopniu potrafią je zaimplementować poprawnie.

- **Minimalne wymagania formalne** — TSP nie wymaga rozbudowanych struktur wejściowych ani specyficznych bibliotek zewnętrznych. Dane wejściowe można reprezentować jako macierz kosztów, a rozwiązanie jako tablicę odwiedzanych wierzchołków, co sprzyja jednoznacznej ocenie poprawności i ułatwia automatyzację testowania.

- **Możliwość automatycznej weryfikacji wyników** — dla wielu instancji problemu znane są dokładne lub przybliżone wyniki, co umożliwia obiektywną ocenę wygenerowanego kodu bez potrzeby każdorazowej analizy manualnej.

- **Analiza wielu aspektów kodu** — kod rozwiązujący TSP można oceniać nie tylko pod kątem poprawności i efektywności, ale także pod względem jakości strukturalnej (czytelność, złożoność cyklomatyczna, liczba ostrzeżeń w analizie statycznej), co dobrze wpisuje się w zastosowanie narzędzi takich jak SonarQube czy IntelliJ IDEA.

3.1.3 Rola TSP jako reprezentanta złożonych problemów algorytmicznych

Choć TSP nie jest jedynym problemem o wysokiej złożoności algorytmicznej, jego wybór wynika również z faktu, że jest powszechnie rozpoznawalny i szeroko reprezentowany w zbiorach treningowych modeli językowych. W związku z tym możliwe jest nie tylko ocenienie czysto „algorytmicznej" jakości rozwiązania, ale również zbadanie, czy narzędzia takie jak ChatGPT czy GitHub Copilot wykorzystują wcześniej poznane wzorce, czy też konstruują kod od podstaw w sposób kreatywny i dostosowany do zadanego kontekstu.

W eksperymencie przedstawionym w dalszej części pracy, TSP stanowi wspólną oś testową dla wszystkich modeli, umożliwiając rzetelną ocenę ich możliwości i ograniczeń. Dzięki różnym wariantom instancji (różne rozmiary grafów, wersje symetryczne i asymetryczne), możliwe jest również zróżnicowanie poziomu trudności i precyzyjna analiza wpływu złożoności na jakość wygenerowanego kodu.

3.2 Charakterystyka porównywanych narzędzi AI

W ramach eksperymentu porównano możliwości siedmiu narzędzi opartych na dużych modelach językowych (LLM), dostępnych publicznie w marcu 2025 roku. Celem było sprawdzenie, jak narzędzia te radzą sobie z generowaniem rozwiązań algorytmicznych na przykładzie problemu komiwojażera (TSP), przy wykorzystaniu spójnego zestawu zapytań. Wszystkie narzędzia (z wyjątkiem Tabnine) były testowane w formie czatu za pomocą przeglądarki internetowej, z wykorzystaniem domyślnych interfejsów użytkownika udostępnianych przez producentów. W przypadku Tabnine wykorzystano wtyczkę do edytora Visual Studio Code, która oferuje interfejs czatu wbudowany w edytor.

3.2.1 Lista użytych modeli i środowisk

W ramach eksperymentu porównano siedem narzędzi opartych na dużych modelach językowych (LLM), służących do generowania kodu źródłowego.

- **ChatGPT (OpenAI)** ChatGPT [24] to chatbot oparty na rodzinie modeli GPT (Generative Pre-trained Transformer), opracowanych przez firmę OpenAI. Od momentu premiery w listopadzie 2022 r. narzędzie to zdobyło szeroką popularność, a jego interfejs oferuje możliwość interakcji z modelami językowymi poprzez naturalne zapytania tekstowe. Wersja darmowa korzysta z modelu **GPT-3.5**, natomiast subskrybenci planu *ChatGPT Plus* mają dostęp do **GPT-4-turbo**. Oba modele wspierają generację kodu w wielu językach programowania, oferując dodatkowe funkcje takie jak formatowanie w Markdown, komentarze w kodzie oraz rozumienie poleceń kontekstowych. Modele są trenowane na danych pochodzących z internetu, w tym z publicznie dostępnych repozytoriów kodu.

- **GitHub Copilot Chat (GitHub & OpenAI)** GitHub Copilot [12] to narzędzie do wspomagania kodowania opracowane przez GitHub we współpracy z OpenAI. Pierwotnie zaprezentowane zostało jako rozszerzenie do edytorów kodu (Visual Studio Code, JetBrains). Copilot od 2023 roku dostępny jest także jako interfejs czatowy o nazwie **Copilot Chat**. Narzędzie to korzysta z modeli językowych z rodziny GPT-4, zoptymalizowanych pod kątem programowania. Umożliwia zadawanie pytań w języku naturalnym, generowanie funkcji i klas, tłumaczenie kodu, a także wyjaśnianie działania fragmentów programu. Zintegrowane jest z ekosystemem GitHub, co pozwala na kontekstowe rozumienie kodu zawartego w repozytoriach.

- **Microsoft Copilot (Copilot for Web, dawniej Bing Chat)** Microsoft Copilot [22] to ogólny interfejs konwersacyjny oparty na GPT-4-turbo, będący częścią pakietu usług Microsoft 365. Copilot oferuje tryby interakcji takie jak *Balanced*, *Creative* i *Precise*, co wpływa na styl generowanych odpowiedzi. Choć nie jest

narzędziem stricte programistycznym, wspiera generowanie kodu źródłowego, analizę fragmentów kodu oraz tłumaczenia między językami programowania. Integracja z wyszukiwarką Bing oraz usługami Microsoft pozwala na dostęp do zaktualizowanych informacji oraz dokumentacji technicznej.

- **Google Gemini (dawniej Bard)** Gemini [14] to produkt firmy Google w dziedzinie generatywnej sztucznej inteligencji, który w grudniu 2023 r. zastąpił wcześniejsze narzędzie Bard. Gemini dostępny jest w wersji przeglądarkowej i obsługuje wiele zadań językowych, w tym programowanie. Na potrzeby generowania kodu Gemini wykorzystuje model **Gemini 1.5 Pro**. Model potrafi generować kod w wielu językach, analizować błędy oraz proponować alternatywne implementacje. Wyróżnia się integracją z wyszukiwarką Google oraz usługami chmurowymi, co umożliwia łatwy dostęp do dokumentacji technicznej.

- **Claude 3 (Anthropic)** Claude [1] to seria modeli językowych rozwijanych przez firmę Anthropic, skoncentrowana na „konstytucyjnym AI" – metodzie trenowania, w której zamiast opierać się wyłącznie na klasycznym uczeniu ze wzmacnianiem na podstawie ludzkiej informacji zwrotnej, model kierowany jest zestawem jawnie zdefiniowanych zasad, swoistą „konstytucją". Zasady te obejmują m.in. dążenie do bycia pomocnym, nieszkodliwym i uczciwym, a model uczy się samodzielnie oceniać i poprawiać swoje odpowiedzi w oparciu o te wytyczne. Takie podejście ma zapewnić większą spójność, bezpieczeństwo i przewidywalność zachowania systemu. W momencie wykonywania testów w marcu 2025 roku dostępny był model **Claude 3 Opus**, udostępniony bezpłatnie. Claude obsługuje szeroki zakres zadań, w tym programowanie. Wspiera analizę kodu, generowanie funkcji i klas, a także tłumaczenie i poprawianie błędów. Model cechuje się długim kontekstem (do 200 tysięcy tokenów) oraz zaawansowaną kontrolą struktury wypowiedzi, co ułatwia pracę z rozbudowanymi fragmentami kodu.

- **DeepSeek Coder** DeepSeek [9] to otwarty projekt badawczy skoncentrowany na zastosowaniach AI w programowaniu. Wersja publiczna z marca 2025 r. oferowała dostęp do modelu **DeepSeek Coder 6.7B** przez interfejs webowy. Model ten trenowany był w dużej mierze na danych kodowych (głównie z GitHub), z ukierunkowaniem na dokładność generacji i zgodność składniową. DeepSeek potrafi generować kod z uwzględnieniem promptów tekstowych, często proponując kilka możliwych podejść do danego problemu. Wyróżnia się szybkością działania i dużą efektywnością w kontekście typowych zadań algorytmicznych.

- **Tabnine** Tabnine [30] to narzędzie wspomagające kodowanie oparte na dużych modelach językowych, zaprojektowane specjalnie do zastosowań programistycznych. Początkowo funkcjonowało jako system predykcyjny integrujący się z edytorami

kodu (takimi jak Visual Studio Code, JetBrains czy Neovim), podpowiadający kolejne fragmenty kodu na podstawie kontekstu. Od 2023 roku Tabnine oferuje również funkcję **Tabnine Chat** – interaktywnego czatu z modelem AI, działającego analogicznie do GitHub Copilot Chat czy ChatGPT, z naciskiem na zadania kodowe. Tabnine Chat jest zintegrowany z edytorem kodu i pozwala użytkownikowi na zadawanie pytań, generowanie fragmentów kodu, wyjaśnianie działania algorytmów oraz refaktoryzację. Wersja wykorzystana w niniejszej pracy pochodziła z płatnego planu *Tabnine Pro*, który opiera się na własnym modelu LLM trenowanym wyłącznie na danych w postaci kodu źródłowego – bez udziału tekstów naturalnych spoza repozytoriów. Model został zaprojektowany z myślą o prywatności oraz wydajności – możliwa jest zarówno praca z użyciem chmury Tabnine, jak i wdrożeń lokalnych (on-premise). W odróżnieniu od narzędzi ogólnego przeznaczenia, Tabnine nie jest wykorzystywany do typowych zadań konwersacyjnych, a jego odpowiedzi są zazwyczaj precyzyjnie ukierunkowane na kontekst kodu źródłowego.

3.2.2 Wersje modeli i tryb dostępu

Wszystkie narzędzia (z wyjątkiem Tabnine) były testowane w swoich darmowych wersjach, dostępnych bezpośrednio w przeglądarce. Wersje modeli, z których korzystano:

- **ChatGPT** – model GPT-3.5 (darmowy), GPT-4-turbo dostępny wyłącznie w subskrypcji ChatGPT Plus.
- **GitHub Copilot Chat** – wersja przeglądarkowa bez dodatkowych integracji z IDE, korzystająca z modelu opartego na GPT-4 (dokładna wersja nieujawniona).
- **Microsoft Copilot** – model GPT-4-turbo dostępny w darmowej wersji w ramach usługi copilot.microsoft.com.
- **Gemini** – wersja Gemini 1.5 Pro, dostępna publicznie bez opłat.
- **Claude 3.0 Opus** – darmowa wersja oferowana przez Anthropic.
- **DeepSeek Coder** – wersja 6.7B (domyślna), z opcją przełączenia na model 33B (w badaniu użyto modelu domyślnego).
- **Tabnine** – wersja Tabnine Pro, model płatny (brak wersji darmowej).

3.2.3 Zasady generowania rozwiązań i konstrukcja zapytań

Inżynieria zapytań – znaczenie i założenia Współczesne modele językowe dużej skali są silnie zależne od sposobu formułowania zapytań. Inżynieria zapytań, czyli projektowanie skutecznych zapytań do modeli generatywnych, stanowi dziedzinę samą w sobie —

wymagającą zrozumienia nie tylko języka naturalnego, ale i wewnętrznych mechanizmów działania modelu.

W niniejszej pracy przyjęto podejście odwzorowujące dwa najbardziej typowe scenariusze interakcji programisty z asystentem AI:

- Prompt ogólny — odpowiadający sytuacji, w której użytkownik prosi narzędzie o „pełne rozwiązanie" danego problemu, bez uprzedniego narzucania struktury.
- Prompt kontekstowy — bliższy sytuacji, gdy programista korzysta z LLM w ramach istniejącej architektury projektu i oczekuje dopasowania rozwiązania do konkretnego interfejsu lub konwencji kodu.

W celu zapewnienia spójności oraz umożliwienia rzetelnego porównania odpowiedzi generowanych przez różne modele językowe, zaprojektowano dwa warianty zapytań:

- **Zapytanie ogólne**:

  ```
  Create a TSP solution in Java
  ```

  Zapytanie o charakterze minimalistycznym, mające na celu ocenę zdolności modelu do samodzielnej interpretacji problemu komiwojażera i zaproponowania pełnej implementacji na podstawie jedynie ogólnej specyfikacji celu.

- **Zapytanie kontekstowe (ze specyfikacją interfejsu i klas)**:

  ```
  Create a class that would implement solveTSP method in this
  interface:

  public interface TspAlgorithm {
      TspResult solveTSP(int[][] graph);
  }

  The TspResult class consists of these fields:

  public class TspResult {
      private final int[] path;
      private final int cost;

      public TspResult(int[] path, int cost) {
          this.path = path;
          this.cost = cost;
  ```

```
    }

    public int[] getPath() {
        return path;
    }

    public int getCost() {
        return cost;
    }
}
```

Prompt zawierający szczegółowy kontekst programistyczny, w tym definicję interfejsu oraz klasy pomocniczej. Celem było sprawdzenie, jak modele radzą sobie z bardziej restrykcyjnym i jednoznacznym opisem wymagań implementacyjnych.

Wszystkie odpowiedzi były kopiowane bez modyfikacji do środowiska IntelliJ IDEA, a przypadkach, gdy model zwrócił więcej niż jedno rozwiązanie (np. DeepSeek – wariant brute-force i heurystyczny), każde z nich analizowano oddzielnie, z zachowaniem identycznych kryteriów oceny.

3.3 Struktura testów funkcjonalnych

W celu obiektywnej oceny jakości i poprawności rozwiązań generowanych przez modele językowe, opracowano zbiór testów jednostkowych w języku Java, wykorzystujących bibliotekę `JUnit 5` oraz mechanizmy testów parametrycznych. Testy te pozwalają automatycznie weryfikować zachowanie implementacji algorytmu komiwojażera względem oczekiwanych rezultatów, zarówno pod względem funkcjonalnym, jak i odporności na niepoprawne dane wejściowe.

3.3.1 Organizacja kodu testowego

Wszystkie testy implementowano w klasie `TspTest`, przy czym do uruchamiania wielu implementacji tego samego interfejsu wykorzystano mechanizm `@ParameterizedTest` z dostawcą argumentów w klasie `TspSolverProvider`:

```
1 return Stream.of(
2     Arguments.of("ChatGPT", new ComplexChatGptSolution()),
3     Arguments.of("MS Copilot", new ComplexMsCopilotSolution()),
4     Arguments.of("Github Copilot", new
          ComplexGithubCopilotSolution()),
```

```
5       Arguments.of("Claude", new ComplexClaudeSolution()),
6       Arguments.of("DeepSeek BruteForce", new
            ComplexDeepSeekBruteForceSolution()),
7       Arguments.of("DeepSeek Nearest Neighbour", new
            ComplexDeepSeekNNSolution()),
8       Arguments.of("Google Gemini", new ComplexGeminiSolution()),
9       Arguments.of("Tabnine", new ComplexTabnineSolution())
10  );
```

Każda z powyższych klas implementuje interfejs `TspAlgorithm`, co pozwala na zunifikowane podejście do testowania wszystkich rozwiązań.

3.3.2 Przykład instancji testowej (4x4)

Dla celów walidacyjnych stworzono kilka statycznych instancji grafów testowych, zdefiniowanych w klasie pomocniczej `GraphUtils`. Poniżej przykład grafu 4-wierzchołkowego (symetrycznego), użytego w testach:

$$G = \begin{bmatrix} 0 & 10 & 15 & 20 \\ 10 & 0 & 35 & 25 \\ 15 & 35 & 0 & 30 \\ 20 & 25 & 30 & 0 \end{bmatrix}$$

Oczekiwana optymalna trasa: $0 \to 1 \to 3 \to 2 \to 0$ Koszt całkowity: $10+25+30+15 = 80$

Kod testu dla tej instancji:

```
1  GraphUtils.SampleTspData data = GraphUtils.getSampleGraph4x4();
2
3  int[][] graph = data.getGraph();
4  TspResult expected = data.getExpectedResult();
5
6  TspResult actual = solver.solveTSP(graph);
7
8  assertTrue(GraphUtils.validatePath(actual.getPath(), graph.
       length), solverName + ": Invalid path!");
9  assertEquals(expected.getCost(), actual.getCost(), solverName +
       ": Incorrect cost!");
```

3.3.3 Testy instancji o zwiększonej złożoności

Dla instancji o większym rozmiarze (10x10, 15x15) testy dodatkowo rejestrują czas wykonania i zużycie pamięci operacyjnej:

```
Runtime runtime = Runtime.getRuntime();
runtime.gc();
long beforeUsedMem = runtime.totalMemory() - runtime.freeMemory
    ();

long startTime = System.nanoTime();
TspResult actual = solver.solveTSP(graph);
long endTime = System.nanoTime();

long afterUsedMem = runtime.totalMemory() - runtime.freeMemory()
    ;
long timeMillis = (endTime - startTime) / 1_000_000;
long memKB = (afterUsedMem - beforeUsedMem) / 1024;
```

Wszystkie implementacje są oceniane pod względem:

- poprawności wygenerowanej ścieżki,
- zgodności kosztu z oczekiwanym (najkrótszym znanym rozwiązaniem),
- stabilności wykonania (brak błędów uruchomieniowych),
- wydajności (czas i pamięć).

3.3.4 Testy odporności na błędne dane wejściowe

Duży nacisk położono na testowanie odporności rozwiązań na dane wejściowe niezgodne ze specyfikacją. Dla każdego solvera sprawdzano, czy generowany kod poprawnie obsługuje przypadki wyjątkowe:

- `null` jako wejściowa macierz grafu:

```
    assertThrows(IllegalArgumentException.class, () ->
        solver.solveTSP(null));
```

- Pusta macierz:

```
    int[][] emptyGraph = new int[0][0];
    assertThrows(IllegalArgumentException.class, () ->
        solver.solveTSP(emptyGraph));
```

- Macierz niekwadratowa:

```
int[][] nonSquareGraph = {
    {0, 10, 15},
    {10, 0, 20}
};
assertThrows(IllegalArgumentException.class, () ->
    solver.solveTSP(nonSquareGraph));
```

- Macierz zawierająca wiersze o wartości `null`:

```
int[][] badGraph = new int[3][];
badGraph[0] = new int[]{0, 1, 2};
badGraph[1] = null;
badGraph[2] = new int[]{2, 1, 0};
assertThrows(IllegalArgumentException.class, () ->
    solver.solveTSP(badGraph));
```

3.3.5 Walidacja ścieżki i kosztu

Dla każdej odpowiedzi zwracanej przez solver zastosowano dwa niezależne mechanizmy walidacji:

1. **Ścieżka**: musi zawierać wszystkie wierzchołki dokładnie raz (poza punktem powrotu) i rozpoczynać się oraz kończyć w tym samym mieście. Walidacja wykonywana przez metodę:

```
GraphUtils.validatePath(int[] path, int expectedLength)
```

2. **Koszt**: porównywany z wartością oczekiwaną, znaną z optymalnego rozwiązania.

3.3.6 Zasady zaliczenia testów

Rozwiązanie uznawano za poprawne, jeśli:

- nie zawierało błędów składniowych (kompilowało się w środowisku IntelliJ IDEA),
- implementowało wymagany interfejs (`TspAlgorithm`),
- zwracało poprawną ścieżkę (cykl Hamiltona),
- poprawnie zgłaszało wyjątki w testach negatywnych,
- nie powodowało błędów wykonania.

Dzięki ujednoliconej strukturze testów oraz automatyzacji procesu, możliwe było efektywne porównanie wszystkich rozwiązań w sposób powtarzalny, bez ręcznej ingerencji.

3.4 Środowisko testowe i narzędzia

W celu przeprowadzenia badań nad jakością i poprawnością kodu wygenerowanego przez modele językowe, zastosowano dwa główne narzędzia wspomagające analizę i testowanie oprogramowania: środowisko programistyczne IntelliJ IDEA oraz platformę SonarCloud (chmurową wersję narzędzia SonarQube). Oba rozwiązania odegrały istotną rolę w wykrywaniu problemów technicznych, analizie struktury kodu oraz ocenie jego czytelności i utrzymywalności.

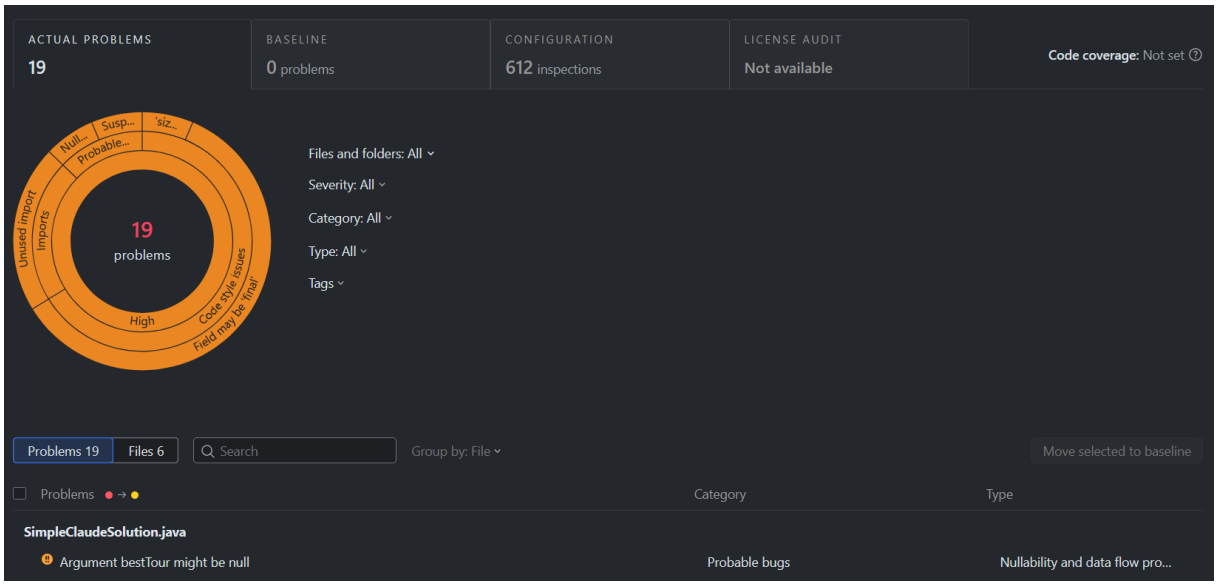
3.4.1 Środowisko IDE – IntelliJ IDEA

Wszystkie testowane rozwiązania implementowano i uruchamiano w środowisku **IntelliJ IDEA** firmy JetBrains. To jedno z najczęściej wykorzystywanych środowisk dla języka Java, znane z rozbudowanej integracji narzędzi wspierających jakość kodu oraz pracy w zespołach programistycznych.

Najważniejsze funkcjonalności używane w badaniu:

- **Zaawansowana analiza składni i semantyki:** IntelliJ automatycznie wykrywa błędy kompilacji, nieużywane zmienne, problemy z inicjalizacją, a także błędy typowe dla Javy, np. nieprzypisane wartości, ryzyko null pointer exception (NPE), czy nieobsłużone wyjątki.
- **Mechanizm sugestii (code inspections):** IntelliJ oferuje system podpowiedzi, który automatycznie wskazuje fragmenty kodu wymagające refaktoryzacji lub mogące prowadzić do błędów logicznych. Inspekcje te obejmują m.in. zbyt długie metody, nieczytelne warunki logiczne, zduplikowany kod czy zbędne konstrukcje pętli.
- **Wsparcie dla testów jednostkowych:** Pełna integracja z biblioteką `JUnit 5`, umożliwiająca uruchamianie testów bezpośrednio z poziomu edytora i podgląd wyników (czas, logi, błędy).
- **Refaktoryzacja i automatyczne poprawki:** Narzędzie wspiera automatyczną poprawę kodu (Quick Fix), co ułatwia eliminację błędów syntaktycznych i konfiguracyjnych bez wpływu na logikę działania.

Zastosowanie Qodany Dodatkowo, IntelliJ IDEA została rozszerzona o narzędzie **Qodana** – statyczny analizator kodu stworzony, podobnie jak środowisko IntelliJ, przez firmę



Rysunek 3.1: Przykładowy raport wygenerowany przez narzędzie Qodana.

JetBrains. Qodana umożliwia przeprowadzenie zautomatyzowanej analizy kodu w formie niezależnego raportu — co pozwala na integrację z pipeline'ami CI/CD lub ocenę jakości poza kontekstem samego IDE. Przykładowy raport wygenerowany przez Qodanę został przedstawiony na Rys. 3.1.

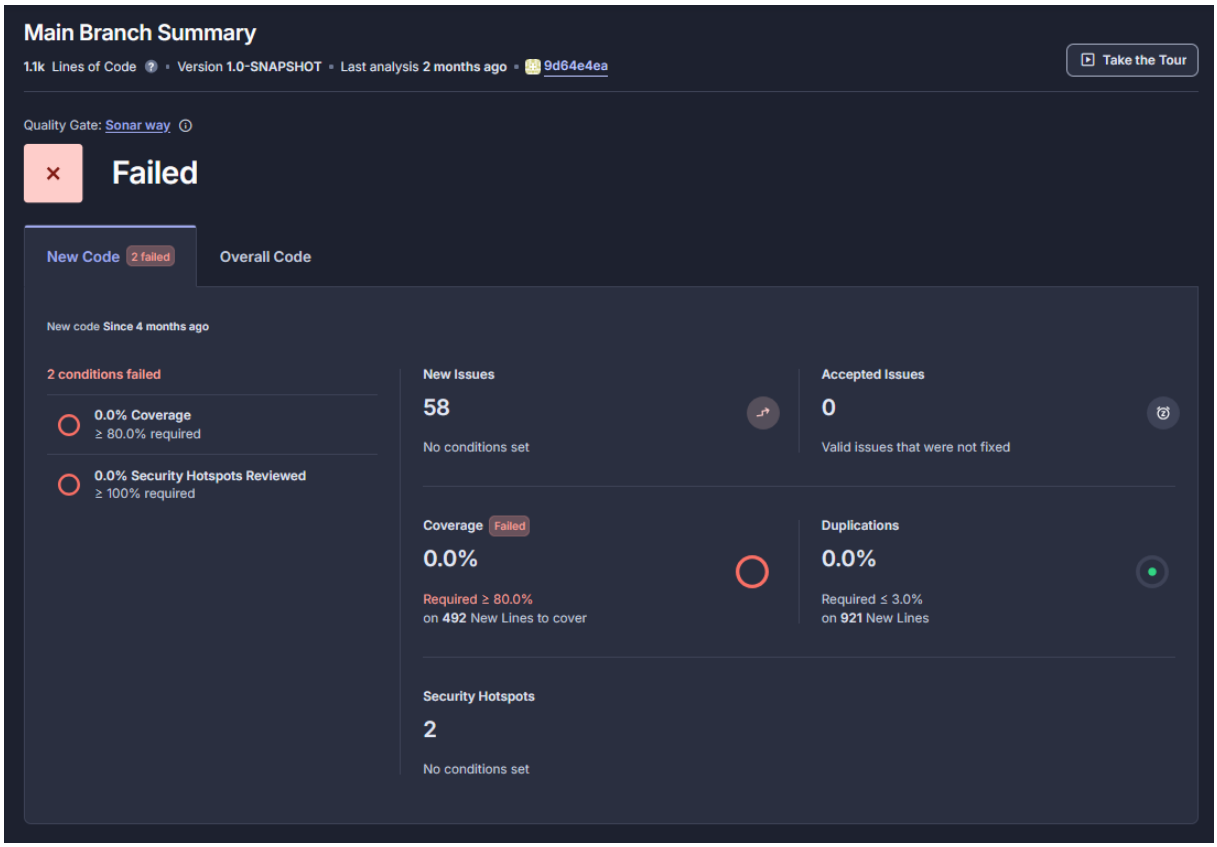
W ramach badań, Qodana została użyta jako źródło dodatkowych uwag dotyczących spójności formatowania, stylu kodu oraz złożoności logicznej.

3.4.2 Analiza jakości kodu – SonarQube (SonarCloud)

Czym jest statyczna analiza kodu? Statyczna analiza kodu (ang. *static code analysis*) to proces przetwarzania kodu źródłowego w celu wykrycia problemów logicznych, stylistycznych i strukturalnych — bez potrzeby jego kompilacji czy uruchomienia. Jej celem jest:

- zwiększenie czytelności i przewidywalności kodu,
- wykrycie tzw. „code smells" – wzorców, które nie są błędami, ale wskazują na złą strukturę lub potencjalne problemy,
- zapewnienie zgodności z przyjętymi standardami kodowania,
- wczesne wykrywanie błędów mogących objawić się dopiero w czasie działania aplikacji.

Aby ocenić jakość strukturalną kodu generowanego przez modele LLM, zastosowano narzędzie **SonarQube**, w wersji chmurowej jako **SonarCloud**. Jest to jedno z najbardziej rozbudowanych narzędzi do *statycznej analizy kodu*. Narzędzie analizuje składnię, strukturę oraz wzorce kodu w celu wykrycia:



Rysunek 3.2: Przykładowy raport wygenerowany przez narzędzie SonarQube.

- potencjalnych błędów wykonania,
- nieczytelnych lub trudnych w utrzymaniu fragmentów,
- wzorców prowadzących do podatności bezpieczeństwa,
- naruszeń zasad dobrych praktyk programistycznych.

SonarQube korzysta z własnego zestawu reguł oraz tzw. Quality Gates — progów oceny jakości kodu, które można konfigurować w zależności od wymagań projektu. Przykładowy raport wygenerowany przez SonarQube przedstawiono na Rys. 3.2.

SonarCloud – szczegóły wdrożenia i analizy W ramach tej pracy wykorzystano **SonarCloud**, który został skonfigurowany do analizy repozytorium git zawierającego wszystkie wygenerowane rozwiązania TSP. Analiza była uruchamiana przy użyciu narzędzia `sonar-scanner` w konfiguracji lokalnej, a wyniki przesyłane były do publicznego dashboardu SonarCloud.

W raportach skoncentrowano się na zakładce **Issues**, w szczególności na problemach klasyfikowanych jako:

- **Maintainability** – zawiera ostrzeżenia dotyczące struktury kodu, w tym złożoności

warunków logicznych, długości metod, zagnieżdżeń, powtarzalności i braku spójności.

- **Reliability** – problemy mogące skutkować błędami wykonania. W analizowanym kodzie nie zidentyfikowano żadnych krytycznych błędów.
- **Security** – potencjalne zagrożenia bezpieczeństwa. Także w tej sekcji nie stwierdzono żadnych ostrzeżeń.

Typowe metryki i ostrzeżenia analizowane w SonarCloud:

- **Cognitive Complexity** – miara trudności zrozumienia fragmentu kodu.
- **Code Duplication** – identyfikacja powtórzonych fragmentów.
- **Function Complexity** – długość i liczba rozgałęzień w metodzie.
- **Comment Density** – gęstość komentarzy względem kodu.
- **Line Coverage** – wskaźnik pokrycia testami (niezastosowany w tej pracy, ale wspierany przez narzędzie).

Dzięki zastosowaniu SonarCloud możliwe było przeprowadzenie jednolitej, mierzalnej i powtarzalnej oceny kodu wygenerowanego przez każde z testowanych narzędzi AI. Wnioski wynikające z tej analizy zostaną omówione szczegółowo w dalszych rozdziałach pracy.

3.4.3 Pomiar czasu i zużycia pamięci

Wydajność obliczeniowa poszczególnych rozwiązań oceniana była poprzez pomiar:

- **czasu wykonania**: zarejestrowanego przy pomocy `System.nanoTime()` na początku i końcu działania metody `solveTSP()`,
- **zużycia pamięci**: szacowanego jako różnica obciążenia sterty JVM przed i po wywołaniu algorytmu, z użyciem `Runtime.getRuntime()` i manualnego wywołania `gc()` w celu zminimalizowania wpływu zarządzania pamięcią przez JVM.

Wartości te były rejestrowane w kontekście testów instancji 10x10 oraz 15x15 i posłużyły jako wskaźniki efektywności generowanych implementacji.

Rozdział 4

Badania

Celem niniejszego rozdziału jest przedstawienie wyników empirycznej oceny narzędzi do generowania kodu na przykładzie problemu komiwojażera (TSP), z naciskiem na porównanie jakości wytworzonych implementacji oraz wpływu kontekstu zapytania (*promptu*) na uzyskane rezultaty. Rozdział koncentruje się na danych pomiarowych z testów jednostkowych, statycznej analizie kodu oraz — tam, gdzie to właściwe — metrykach wydajnościowych.

Źródła danych i artefakty Wyniki pochodzą z uruchomień kodu w repozytorium *AI-tools-assessment* oraz zebrane są w arkuszu wyników (dwa arkusze: wariant *ograniczonego kontekstu* i wariant *złożonego kontekstu*). Analiza jakości kodu opiera się na: inspekcjach IntelliJ IDEA (wspomaganych przez Qodana), oraz raportach SonarCloud w sekcji *Issues* — z naciskiem na kategorię *Maintainability*. W kategoriach *Reliability* i *Security* nie zaobserwowano problemów. W przypadku metryk wykonania stosuje się czas (ms) mierzony `System.nanoTime()` oraz przyrost pamięci (KB) szacowany poprzez `Runtime.getRuntime()`.

Podział rozdziału Ze względu na odmienny cel analizy w dwóch scenariuszach promptów, rozdział dzieli się na dwie, niezależne części:

- **Analiza wariantu z ograniczonym kontekstem** W tej części oceniany jest przede wszystkim *styl i jakość kodu*:
  - obiektywne miary z SonarCloud: liczba i typy *code smells* (*Maintainability-/Issues*), z uwzględnieniem m.in. złożoności poznawczej, duplikacji i długości metod;
  - inspekcje IntelliJ IDEA / Qodany: ostrzeżenia, zalecenia refaktoryzacyjne, spójność ze standardami Javy;
  - krótkie wstawki kodu ilustrujące typowe wzorce i niedoskonałości (np. konstrukcje pętli, walidacja wejścia, liczenie kosztu).

Celem tej analizy jest izolowana ocena „czystości” i struktury kodu przy minimalnym opisie zadania, gdzie przez czystość rozumiana jest koncepcja *clean code* przedstawiona przez Roberta Martina [21].

- **Analiza wariantu ze złożonym kontekstem** W przypadku tej analizy, ocena jest *pełna*: obok jakości i stylu kodu ujęte są również metryki wykonania oraz wyniki testów funkcjonalnych:
  - poprawność funkcjonalna na instancjach 4x4, 10x10, 15x15 oraz wariant asymetryczny (pass/fail według testów JUnit, odporność na błędne dane wejściowe);
  - metryki wydajnościowe: czas i przyrost pamięci (dla 10x10 i 15x15), wraz z identyfikacją awarii (np. wykroczenie poza zakres tablicy, przepełnienie stosu);
  - statyczna analiza jakości (SonarCloud/IntelliJ) — w tym samym ujęciu co w wariancie ograniczonego kontekstu;
  - krótkie studia przypadków z fragmentami kodu (m.in. warianty DeepSeek: brute-force i *nearest neighbour*).

Forma prezentacji W obu częściach wykorzystywane są zwięzłe tabelaryczne zestawienia (pass/fail, liczność problemów *Maintainability*, adnotacje o nieudanym uruchomieniu), a także wykresy porównawcze (np. rozkład liczby *code smells*, czas/pamięć per narzędzie w wariancie złożonego kontekstu). Dla wybranych narzędzi dołączane są krótkie wstawki kodu (kilka–kilkanaście linii) obrazujące charakterystyczne decyzje implementacyjne.

Ograniczenia i uwagi porządkowe Wyniki odnoszą się do publicznych wersji narzędzi (stan na marzec 2025 r.); dla *DeepSeek* uwzględniono dwa rozwiązania z jednej odpowiedzi (brute-force i *nearest neighbour*). Niezdeterminowana natura LLM oraz zmienność modeli w czasie są naturalnym ograniczeniem — dlatego nacisk położono na standaryzację promptów, testów i kryteriów oceny.

4.1 Analiza rozwiązań dla prostego zapytania (ograniczony kontekst)

4.1.1 Przegląd implementacji uzyskanych za pomocą poszczególnych narzędzi

ChatGPT (`SimpleChatGptSolution`) Implementacja opiera się na przeszukiwaniu z powrotem (rekurencyjne przeszukiwanie w głąb, ang. *depth-first search, DFS*) z klasycznymi strukturami pomocniczymi: bufor ścieżki, znacznik odwiedzin oraz akumulator kosztu. Rozwiązanie systematycznie rozszerza częściowe trasy, aktualizując najlepszy

dotychczasowy wynik. Styl jest zwięzły: sensowne nazewnictwo pól (`distanceMatrix`, `visited`, `bestRoute`), niewiele kodu pomocniczego. Z punktu widzenia analizy statycznej przeważają drobne kwestie jakościowe: bezpośrednie wypisy do konsoli, importy do uporządkowania oraz pola, które mogą zostać zadeklarowane jako niezmienne.

Claude (`SimpleClaudeSolution`) To najbardziej rozbudowana odpowiedź — *dydaktyczny pokaz* wielu podejść w jednej klasie. Oprócz wariantu **siłowego** (generowanie wszystkich permutacji z oceną kosztu) i heurystyki **najbliższego sąsiada**, kod zawiera także:

- **2-opt** — iteracyjne poprawianie trasy poprzez zamianę dwóch krawędzi, gdy skrzyżowania można „rozplątać" i skrócić drogę; pętla poprawiająca działa, dopóki udaje się znaleźć skrócenie.
- **Algorytm genetyczny (GA)** — utrzymywana jest populacja tras, z selekcją, krzyżowaniem (np. warianty *order crossover*) i prostymi mutacjami (zamiana kolejności miast) oraz oceną przystosowania (koszt trasy). W kodzie wyodrębniono konstrukcje generujące populację startową i pętlę ewolucyjną.

Ważny element to czytelny interfejs porównawczy — ta sama klasa wypisuje czasy wykonania i koszty dla kilku metod, co ułatwia szybkie pokazanie różnic w praktyce. Poniżej przedstawiono przykładowy wynik wywołania wygenerowanej przez Claude klasy:

```
=== TSP Solver Demo ===

Cities:
  A(0.0,0.0)
  B(1.0,5.0)
  C(5.0,3.0)
  D(6.0,1.0)
  E(3.0,0.0)
  F(2.0,4.0)

=== Solutions ===

Brute Force Solution:
Distance: 18,07
Time: 3 ms
Tour: A -> B -> F -> C -> D -> E -> A

Nearest Neighbor Solution:
```

```
Distance: 18,07
Time: 0 ms
Tour: A -> E -> D -> C -> F -> B -> A

2-Opt Solution:
Distance: 18,07
Time: 0 ms
Tour: A -> E -> D -> C -> F -> B -> A

Genetic Algorithm Solution:
Distance: 18,07
Time: 100 ms
Tour: A -> E -> D -> C -> F -> B -> A


=== Larger Random Instance (15 cities) ===
Nearest Neighbor: 464,66 (Time: 0 ms)
2-Opt: 362,06 (Time: 0 ms)
Genetic Algorithm: 353,28 (Time: 103 ms)

Improvements over Nearest Neighbor:
2-Opt: 22,1%
Genetic Algorithm: 24,0%
```

Ten format jest świetny edukacyjnie, ale ma konsekwencje utrzymaniowe: duża liczba metod w jednej jednostce, częste wypisy na konsolę, powtarzalne fragmenty oraz miejsca wymagające ostrożności (np. operacje modyfikujące listę wewnątrz pętli). Z perspektywy czytelności nazwy są opisowe, a komentarze wyjaśniają ideę kroków algorytmicznych.

DeepSeek (`SimpleDeepSeekSolution`) Koncepcyjnie zbliżone do ChatGPT: przeszukiwanie z powrotem z pamiętaniem najlepszego kosztu i ścieżki. Struktura jest prosta, z oszczędnymi komentarzami. Najczęstsze uwagi dotyczą bezpośrednich wypisów na konsolę oraz możliwości wzmocnienia niezmienności pól.

GitHub Copilot (`SimpleGithubCopilotSolution`) Implementacja używa podejścia brute force: generuje wszystkie permutacje miast (zwykle przy stałym mieście startowym) i dla każdej oblicza łączny koszt cyklu Hamiltona, zachowując minimum. Złożoność czasowa jest rzędu $O(n! \cdot n)$, a pamięciowa niewielka (głównie bufor bieżącej permutacji i śledzenie najlepszego kosztu). Kod jest zwięzły: prosta pętla/rekurencja do permutacji, jeden kalkulator kosztu i zwrot najlepszego wyniku. Z perspektywy stylu — minimalne

komentarze, czytelne nazwy zmiennych, sporadyczne użycia wyjścia na konsolę (zidentyfikowane przez analizę statyczną) możliwe do zastąpienia loggerem.

Microsoft Copilot (`SimpleMsCopilotSolution`) Również brute force w prostym, kompaktowym wariancie: permutacje trasy + ewaluacja kosztu i domknięcie cyklu do wierzchołka startowego. Struktura funkcji jest bardzo linearna (generuj → policz → porównaj → zapamiętaj), co ułatwia śledzenie przepływu sterowania. Niewielka objętość kodu ogranicza „szum" utrzymaniowy; potencjalne drobiazgi to bezpośrednie wypisy do `System.out` oraz okazje do wzmocnienia niezmienności wybranych pól.

Google Gemini (`SimpleGeminiSolution`) Zastosowano heurystykę najbliższego sąsiada: start z ustalonego miasta, iteracyjne wybieranie najtańszego przejścia do jeszcze nieodwiedzonego wierzchołka i finalne domknięcie cyklu. Złożoność czasowa to w praktyce $O(n^2)$ przy macierzy odległości; rozwiązanie jest szybkie, ale zachłanne i niegwarantujące optimum. Kod jest nieco dłuższy niż w minimalistycznych wariantach brute force, z kilkoma zagnieżdżonymi warunkami i mieszanym użyciem `stdout`/`stderr`; miejscami widać możliwości uproszczeń (np. scalanie warunków) oraz standaryzacji komunikatów na logger.

Tabnine (`SimpleTabnineSolution`) Krótka, implementacja heurystyczna (wariant „najbliższego sąsiada"). Mało kodu, prosty przepływ, pojedyncze naruszenia stylu (final, importy). Dzięki niewielkiej objętości łatwa w lekturze i utrzymaniu.

4.1.2 Wyniki statycznej analizy i metryki czystości kodu

Tabela 4.1 zestawia liczbę uwag z trzech źródeł (SonarCloud, inspekcje IntelliJ IDEA, Qodana) oraz liczbę linii kodu (LOC) dla każdej implementacji wygenerowanej z prostego promptu. Wskaźnik *uwag na 100 LOC* normalizuje wynik względem wielkości pliku.

Narzędzie	SonarCloud	IDEA	Qodana	LOC	Razem	Uwagi/100 LOC
GitHub Copilot	2	0	0	69	2	2,90
Microsoft Copilot	2	0	0	50	2	4,00
Tabnine	2	2	2	73	6	8,22
Claude	31	6	6	458	43	9,39
Google Gemini	11	4	3	148	18	12,16
DeepSeek	9	3	3	121	15	12,40
ChatGPT	5	5	5	73	15	20,55

Tabela 4.1: Zestawienie uwag statycznych i wielkości kodu (ograniczony kontekst)

Ogólny obraz jest spójny z intuicją: krótsze, jednowariantowe implementacje uzyskują bardzo niski współczynnik uwag per 100 LOC. Rozbudowane klasy (Claude) zachowują

akceptowalne proporcje po normalizacji, ale ich *całkowity* koszt utrzymaniowy i złożoność poznawcza rosną wraz z długością pliku i liczbą wariantów.

4.1.3 Najczęstsze kategorie problemów wykryte w analizie

W analizie statycznej (SonarCloud, IntelliJ IDEA, Qodana) powtarzały się głównie:

- **Komunikacja diagnostyczna przez standardowe wyjście** — bezpośrednie użycia `System.out`/`System.err` zamiast loggera.
- **Porządkowanie importów i drobne naruszenia konwencji** — nieużywane importy, proste duplikacje literałów, użycie `size()==0` zamiast `isEmpty()`.
- **Modyfikatory i niezmienność** — pola możliwe do oznaczenia jako *final* lub o zbyt szerokiej widoczności (w sensie hermetyzacji pól).
- **Operacje na kolekcjach** — modyfikacje listy w pętli indeksowanej (ryzyko pominięcia elementu) zamiast iteratora lub `removeIf`.
- **Kontrakty nullowalności** — pojedyncze miejsca, gdzie wynikowa struktura może być niewystarczająco zainicjalizowana przed zwróceniem.

4.1.4 Przykłady i naprawy typowych problemów (z fragmentami kodu)

Zastąpienie `System.out/err` loggerem

Przed:

```
1 System.out.println("Cost = " + bestCost);
2 System.err.println("Invalid path!");
```

Po (przykładowo `java.util.logging`):

```
1 private static final Logger LOG =
2     Logger.getLogger(TspSolver.class.getName());
3 LOG.info(() -> "Cost = " + bestCost);
4 LOG.warning("Invalid path!");
```

Korzyści: możliwość sterowania poziomami logów, wyciszania w testach, spójna telemetria.

Porządkowanie importów

Przed:

```
import java.util.Arrays; // nieuzywany
```

Po:

```
// usunieto nieuzywany import
// IDE moze to zautomatyzowac (funkcja "optimize imports")
```

Korzyści: brak zbędnych zależności w nagłówku, mniejszy „szum” w recenzjach kodu.

Uproszczenie konwencji kolekcji

Przed:

```
if (list.size() == 0) {
    // ...
}
```

Po:

```
if (list.isEmpty()) {
    // ...
}
```

Korzyści: lepsza czytelność, zgodność z idiomami Javy i wskazaniami analizatorów.

Bezpieczne usuwanie elementów listy w pętli

Przed (możliwe pominięcie elementu):

```
for (int i = 0; i < list.size(); i++) {
    if (shouldRemove(list.get(i))) {
        list.remove(i);
    }
}
```

Po (iterator):

```
for (Iterator<Item> it = list.iterator(); it.hasNext();) {
    if (shouldRemove(it.next())) {
        it.remove();
    }
}
```

Lub (Java 8+):

```
list.removeIf(this::shouldRemove);
```

Korzyści: brak niespodziewanego „przeskakiwania” indeksów, poprawna semantyka.

Wzmocnienie niezmienności pól

Przed:

```
private int[][] distanceMatrix;
private int numCities;
```

Po:

```
private final int[][] distanceMatrix;
private final int numCities;
```

Korzyści: jaśniejszy kontrakt klasy, mniej efektów ubocznych, silniejsza analiza przepływu danych.

Kontrakt nullowalności wyniku

Przed (możliwy `null` w argumencie `bestTour`):

```
return new TspResult(bestTour, bestDistance);
```

Po (domyślna inicjalizacja):

```
int[] safeTour = (bestTour != null) ? bestTour : new int[] {0};
return new TspResult(safeTour, bestDistance);
```

Lub z adnotacjami w konstruktorze:

```
public TspResult solveTSP(@NotNull int[] tour, ...) { ... }
```

Korzyści: brak niespodziewanych wyjątków NPE (w przypadku domyślnej inicjalizacji), jawny kontrakt zwracanej wartości dla analizatorów kodu oraz programistów pracujących nad kodem (w przypadku adnotacji).

4.1.5 Podsumowanie rozwiązań dla ograniczonego kontekstu

W trybie ograniczonego kontekstu najczęściej uzyskujemy implementacje algorytmu „najbliższego sąsiada” oraz rozwiązania brute-force'owe. Najbardziej wartościowy dydaktycznie okazał się `SimpleClaudeSolution`, łączący kilka stylów rozwiązań (brute force, NN, 2-opt, GA) z prostą prezentacją porównawczą czasów — materiał do pokazywania kompromisów między jakością a wydajnością. Równocześnie krótsze, jednowariantowe klasy mają lepsze wskaźniki czystości kodu. Wspólne obszary do poprawy to głównie:

eliminacja bezpośredniego `System.out/err`, porządkowanie importów, wzmacnianie niezmienności pól i uważniejsze operacje na kolekcjach.

4.2 Analiza rozwiązań dla złożonego zapytania (pełniejszy kontekst)

4.2.1 Przegląd wygenerowanych rozwiązań

W tej części opisano implementacje wygenerowane dla złożonego wariantu promptu, tj. z precyzyjną specyfikacją interfejsu `TspAlgorithm` i klasy wyniku `TspResult`. Każde rozwiązanie oceniono pod kątem przyjętej strategii algorytmicznej, struktury kodu (modularność, podział na metody pomocnicze), walidacji danych wejściowych oraz podstawowych aspektów stylu i czytelności. Poniższe charakterystyki są oparte na rzeczywistych plikach z pakietu `polsl.tsp.complex`.

ChatGPT (`ComplexChatGptSolution`)

Podejście algorytmiczne: klasyczna enumeracja permutacji miast $[1, \ldots, n-1]$ z „zakotwiczeniem" startu w 0 i domknięciem ścieżki do 0. Dla każdej permutacji budowany jest cykl $[0, p_1, \ldots, p_{n-1}, 0]$, a koszt liczony jest przez proste zsumowanie wag sąsiednich krawędzi. Nie ma przycinania stanów (*branch-and-bound*), heurystyk konstrukcyjnych ani dynamicznego programowania — jest to pełny *brute force*.

Struktura kodu: metoda `solveTSP` przygotowuje listę miast, iteruje po wszystkich permutacjach zwracanych przez `permute`, konstruuje tablicę `path` i aktualizuje `minCost` oraz `bestPath`. Generowanie permutacji realizuje duet metod `permute`/`backtrack`; koszt oblicza prosta pętla w `calculatePathCost`. Nazewnictwo jest rzeczowe, a podział na małe metody poprawia czytelność.

Microsoft Copilot (`ComplexMsCopilotSolution`)

Podejście algorytmiczne: klasyczny *brute force*. Start i koniec trasy są stałe (miasto 0). Następnie rozważane są wszystkie możliwe kolejności odwiedzania pozostałych miast $[1, \ldots, n-1]$. Dla każdej kolejności wyliczany jest koszt przejścia ($0 \to p_1 \to \cdots \to p_{n-1} \to 0$), a najtańsza trasa jest zapamiętywana.

Struktura kodu: metoda `solveTSP` tworzy tablicę `vertices` z numerami miast $[1, \ldots, n-1]$ i w pętli przechodzi przez *wszystkie* możliwe ułożenia tej tablicy. Do generowania „następnej kolejności" służy metoda `nextPermutation`: zamiast używać rekurencji i tworzyć nowe listy, zmienia ona elementy w `vertices` „na miejscu" (czyli bez dodatkowych struktur), tak by otrzymać kolejne ułożenie. W każdej iteracji sumowany jest koszt przejścia od 0, przez miasta w aktualnej kolejności, z powrotem do 0. Jeśli koszt jest lepszy

od dotychczasowego minimum, kod kopiuje bieżącą trasę (dodając 0 na początek i koniec) do `bestPath`. Pomocnicze metody `swap` i `reverse` tylko zamieniają elementy i odwracają końcówkę tablicy — to prosty sposób na wygenerowanie wszystkich możliwych kolejności bez rekurencji i bez dużego zużycia pamięci.

GitHub Copilot (`ComplexGithubCopilotSolution`)

Podejście algorytmiczne: pełne przeszukiwanie przestrzeni rozwiązań metodą rekursywnego *backtrackingu*. Start i koniec trasy są zafiksowane w wierzchołku 0. Rekurencja odwiedza kolejne, jeszcze nieodwiedzone miasta, sumując koszty przejść i — po odwiedzeniu wszystkich — domyka cykl krawędzią (ostatni $\rightarrow$ 0). Nie zastosowano heurystyk konstrukcyjnych ani przycinania gałęzi; ograniczenia wynikają jedynie z tablicy `visited[]` i sprawdzenia istnienia krawędzi.

Struktura kodu: metoda `solveTSP` inicjalizuje wektor `visited` oraz tablicę `path` długości $n{+}1$ (wypełnioną -1) i deleguje właściwą pracę do `tspUtil`. Funkcja `tspUtil` realizuje głębokie przeszukiwanie: przypadek bazowy uruchamia się, gdy licznik odwiedzonych wierzchołków osiąga n i istnieje krawędź powrotna do 0; wówczas trasa jest domykana i zwracany jest łączny koszt. W przypadku ogólnym iterowane są wszystkie miasta; dla kandydatów spełniających warunek „nieodwiedzony i istnieje krawędź" ustawiany jest znacznik w `visited`, do `path` wpisywany jest bieżący wierzchołek, a następnie wywoływana jest rekurencja z uaktualnionym kosztem. Wartość `minCost` akumuluje minimum po wszystkich gałęziach. Całość mieści się w dwóch metodach, z oszczędnym użyciem pomocniczych struktur; kod jest zorientowany na przejrzyste odwzorowanie schematu DFS po permutacjach.

Claude (`ComplexClaudeSolution`)

Podejście algorytmiczne: zachłanne *nearest neighbor*. Punkt startowy jest stały (miasto 0). W każdej iteracji wybierane jest najbliższe nieodwiedzone miasto, trasa jest sukcesywnie wydłużana, a na końcu domykana powrotem do 0. Heurystyka nie gwarantuje optimum i jest wrażliwa na punkt startowy, ale ma niski koszt obliczeń: $\mathcal{O}(n^2)$ dla pełnej macierzy odległości (każdy wybór wymaga liniowego skanu wiersza, wykonywanego ok. n razy).

Struktura kodu: metoda `solveTSP` wykonuje podstawową walidację (null/rozmiar), obsługuje przypadek $n{=}1$, tworzy tablice `visited` oraz `path` długości $n{+}1$. Zmienna `currentCity` jest inicjalizowana na 0, po czym pętla `for` (od $i{=}1$ do $n{-}1$) wywołuje `findNearestCity` w celu wybrania kolejnego wierzchołka; koszt przejścia jest sumowany na bieżąco. Po odwiedzeniu wszystkich miast do `path[n]` wpisywany jest powrót do `path[0]`, a do kosztu dodawana jest krawędź domykająca. Funkcja pomocnicza przeszukuje wiersz `graph[currentCity][*]` i zwraca indeks najbliższego nieodwiedzonego

miasta (przy remisach wybierany jest pierwszy napotkany kandydat). Cały algorytm mieści się w dwóch metodach i operuje na prostych tablicach. Obie metody posiadają również adnotacje Javadoc, a bardziej nieoczywiste fragmenty logiki zawierają pomocne wyjaśniające komentarze.

DeepSeek - wariant 1 (`ComplexDeepSeekBruteForceSolution`)

Podejście algorytmiczne: pełny *brute force* z enumeracją wszystkich permutacji miast $[1, \ldots, n{-}1]$ i domykaniem cyklu do wierzchołka 0. Dla każdej permutacji wyliczany jest koszt trasy $(0 \to p_1 \to \cdots \to p_{n-1} \to 0)$, a następnie wybierana jest trasa o minimalnym koszcie. Czasowo rozwiązanie ma rząd $\mathcal{O}(n! \cdot n)$. Wariant implementacyjny użyty tutaj gromadzi wszystkie permutacje w pamięci, co powoduje bardzo duży narzut pamięciowy.

Struktura kodu: w `solveTSP` wykonywana jest podstawowa walidacja macierzy (`null`/pusta macierz/kwadratowość) i obsługa przypadku $n{=}1$. Lista `cities` zawiera indeksy miast od 1 do $n{-}1$. Metoda `generatePermutations` zwraca *wszystkie* permutacje jako `List<List<Integer>>`, po czym pętla zewnętrzna iteruje po tych permutacjach, liczy koszt przejścia liniową pętlą po wierzchołkach oraz domyka cykl z powrotem do 0. Najlepsza trasa jest zapisywana do `minPath` długości $n{+}1$ (z pozycjami $[0, p_1, \ldots, p_{n-1}, 0]$) wraz z odpowiadającym jej `minCost`. Generowanie permutacji realizowane jest przez duet metod `generatePermutations`/`permute`. Rozwiązanie jest zwarte, oddziela odpowiedzialności (walidacja, generowanie, ocena), ale przez materializowanie wszystkich permutacji preferuje prostotę nad efektywność pamięciową.

DeepSeek - wariant 2 (`ComplexDeepSeekNNSolution`)

Podejście algorytmiczne: zachłanna heurystyka *nearest neighbor*. Start w mieście 0; w każdej iteracji wybierane jest najbliższe nieodwiedzone miasto (minimalna waga wiersza macierzy od bieżącego miasta), aż do odwiedzenia wszystkich wierzchołków. Na końcu ścieżka jest domykana powrotem do 0. Brak strategii rozstrzygania remisów (tie–breaking) i brak fazy poprawy (np. 2-opt).

Struktura kodu: w `solveTSP` jest wstępna walidacja macierzy (null/pusta macierz/kwadratowość) oraz przypadek brzegowy $n{=}1$. Tworzony jest wektor `visited` i tablica `path` długości $n{+}1$ ($[0, \ldots, 0]$). Pętla główna utrzymuje `currentCity`, a wybór kolejnego węzła delegowany jest do `findNearestUnvisitedCity`, która liniowo przeszukuje wiersz `graph[currentCity]` i zwraca indeks nieodwiedzonego miasta o najmniejszym koszcie. Po każdej decyzji aktualizowany jest `totalCost`, oznaczane jest odwiedzenie, a po zakończeniu dodawany jest koszt krawędzi powrotu do 0 i ustawiane `path[n] = 0`. Implementacja jest krótka i czytelna, sporadycznie wykorzystuje krótkie komentarze.

Google Gemini (`ComplexGeminiSolution`)

Podejście algorytmiczne: pełny *brute force* z „zakotwiczeniem" miasta startowego w 0. Rozważane są wszystkie permutacje wierzchołków $[1, \ldots, n-1]$; dla każdej z nich budowany jest cykl $[0, p_1, \ldots, p_{n-1}, 0]$, a następnie wyliczany jest łączny koszt przejścia. Implementacja dopuszcza ideę „braku krawędzi" poprzez ujemną wagę: jeśli w ścieżce pojawi się krawędź o koszcie < 0, trasa jest uznawana za nieważną i pomijana.

Struktura kodu: metoda `solveTSP` przeprowadza podstawową walidację (null/pusta macierz) oraz obsługuje przypadek brzegowy $n{=}1$. Tworzona jest lista `nodesToPermute` z miastami $[1, \ldots, n-1]$, po czym `generatePermutations` zwraca listę wszystkich permutacji (rekurencyjne wyjmowanie pierwszego elementu i wstawianie go we wszystkie pozycje we wszystkich podpermutacjach). Dla każdej permutacji budowana jest tablica `currentPath` długości $n{+}1$ (z domknięciem do 0), a następnie w pętli liniowej akumulowany jest koszt; jeśli po drodze natrafiona zostanie krawędź o koszcie ujemnym, ścieżka jest odrzucana. Najlepsza trasa i jej koszt (`minCost` typu `long`) są aktualizowane, a po zakończeniu zwracany jest `TspResult` (z konwersją kosztu do `int`). Organizacja kodu jest przeciętna - główna metoda `solveTSP` jest dosyć długa, co ogranicza jej czytelność. Zaletą natomiast jest dobre użycie komentarzy.

Tabnine (`ComplexTabnineSolution`)

Podejście algorytmiczne: zachłanny algorytm *najbliższego sąsiada* (nearest neighbor) zakotwiczony w wierzchołku 0. Strategia wyboru przy remisie jest deterministyczna (pierwszy napotkany minimum).

Struktura kodu: metoda `solveTSP` weryfikuje wejście (niepusty, kwadratowy graf), inicjuje `path` długości $n{+}1$ (z miejscem na powrót do startu) oraz `visited`. Budowa trasy rozpoczyna się w węźle 0 i przebiega w pętli `for` od `i=1` do `n-1`; w każdej iteracji wywoływana jest metoda pomocnicza `findNearestUnvisitedCity`, która liniowo przeszukuje wiersz macierzy kosztów dla bieżącego miasta i zwraca indeks najbliższego nieodwiedzonego wierzchołka. Koszt całkowity `totalCost` jest inkrementowany o wagę krawędzi przejścia, a na końcu dodawany jest koszt powrotu `graph[currentCity][0]`. Zwracany jest `TspResult` z kompletnym cyklem $[0, \ldots, 0]$ i policzonym kosztem.

Podsumowanie

W implementacjach dla złożonego kontekstu dominują dwa: pełna enumeracja permutacji (*brute force*) oraz zachłanny algorytm *najbliższego sąsiada* (NN). Pierwsze gwarantuje wynik optymalny kosztem wykładniczej złożoności $\mathcal{O}(n!)$ i pojawia się w wariantach iteracyjnych (generator następnej permutacji) oraz rekursywnych (backtracking). Drugie daje lepszą złożoność $\mathcal{O}(n^2)$ i bardzo prostą implementację, ale bez gwarancji optymalności. Wygenerowany kod rzadko stosuje przycinanie przestrzeni poszukiwań, ulepsze-

nia lokalne (np. 2-opt) czy metody dynamiczne/metaheurystyczne (Held–Karp, SA, GA, ACO). Taki rozkład wskazuje, że modele — przy braku dodatkowych wskazówek — preferują rozwiązania „kanoniczne": albo najprostsze dokładne, albo najprostsze heurystyczne, co upraszcza implementację i poprawia czytelność, lecz ogranicza skalowalność i jakość dla większych instancji.

4.3 Wyniki statycznej analizy kodu (złożone zapytanie)

Wyniki statycznej analizy kodu dla rozwiązań w przypadku złożonego zapytania zostały podsumowane w tabeli 4.2.

Narzędzie	SonarCloud	IDEA	Qodana	LOC	Razem	Uwagi/100 LOC
ChatGPT	0	0	0	64	0	0,0
Microsoft Copilot	0	0	0	71	0	0,0
GitHub Copilot	0	0	0	38	0	0,0
Claude	1	1	1	79	3	3,8
DeepSeek — BruteForce	0	0	0	72	0	0,0
DeepSeek — NN	0	0	0	53	0	0,0
Gemini	2	1	1	95	4	4,2
Tabnine	1	1	1	51	3	5,9

Tabela 4.2: Statyczna analiza klas wygenerowanych na podstawie złożonego zapytania. *Razem = SonarCloud + IDEA + Qodana.*

Spostrzeżenia

- W pakiecie `complex` przeważają drobiazgi stylistyczne: głównie zbędne importy. Nie odnotowano problemów z bezpieczeństwem ani potencjalnych wyjątków w przepływie danych.
- Jedyne poważniejsze ostrzeżenie to podwyższona złożoność kognitywna w jednej metodzie generującej permutacje w `ComplexGeminiSolution` (warto rozważyć uproszczenie/rozbicie na mniejsze kroki).
- Czyste wyniki (0 problemów) uzyskały ChatGPT, Microsoft Copilot, GitHub Copilot oraz oba warianty DeepSeek (`BruteForce` i `NN`).

4.4 Analiza poprawności wygenerowanych rozwiązań

W tej sekcji omówiono poprawność wygenerowanych rozwiązań. Poniższe tabele przedstawiają wyniki testów poprawności na czterech różnych instancjach (tabela 4.3: macierz

odległości 4×4, tabela 4.4: 10×10, tabela 4.5: 15×15, oraz tabela 4.6: instancja grafu asymetrycznego).

macierz 4x4	**Wykonanie**	**Wynik poprawny**	**Najkrótsza ścieżka**
ChatGPT	**tak**	**tak**	**tak**
Claude	**tak**	**tak**	**tak**
DeepSeek — BruteForce	**tak**	**tak**	**tak**
DeepSeek — NN	**tak**	**tak**	**tak**
GitHub Copilot	**nie**	**nie**	**nie**
Microsoft Copilot	**tak**	**nie**	**nie**
Gemini	**tak**	**tak**	**tak**
Tabnine	**tak**	**tak**	**tak**

Tabela 4.3: Poprawność działania — macierz 4×4.

macierz 10x10	**Wykonanie**	**Wynik poprawny**	**Najkrótsza ścieżka**
ChatGPT	**tak**	**tak**	**tak**
Claude	**tak**	**tak**	**nie**
DeepSeek — BruteForce	**tak**	**tak**	**tak**
DeepSeek — NN	**tak**	**tak**	**nie**
GitHub Copilot	**nie**	**nie**	**nie**
Microsoft Copilot	**tak**	**nie**	**nie**
Gemini	**tak**	**tak**	**tak**
Tabnine	**tak**	**tak**	**nie**

Tabela 4.4: Poprawność działania — macierz 10×10.

Testy uznają rozwiązanie za poprawne tylko wtedy, gdy spełnione są oba warunki: (1) ścieżka jest permutacją wszystkich wierzchołków domkniętą do miasta startowego (walidator ścieżki) oraz (2) koszt ścieżki dokładnie pokrywa się z kosztem referencyjnym (optymalnym). Stąd:

- podejścia dokładne na małych instancjach (4×4, 10×10) zwykle zaliczają testy (prawidłowa ścieżka i koszt optymalny),
- heurystyka NN zwraca cykl poprawny strukturalnie, lecz często nie otrzymuje kosztu referencyjnego.

Skalowanie do 15×15

Dla największej instancji podejścia oparte na pełnym przeglądzie nie dostarczają wyniku zgodnego z kryterium testu (brak zaliczenia poprawności), natomiast NN zwraca trasę, ale — jako heurystyka — nie osiąga kosztu optymalnego, więc również nie przechodzi asercji na koszt.

macierz 15x15	Wykonanie	Wynik poprawny	Najkrótsza ścieżka
ChatGPT	**nie**	**nie**	**nie**
Claude	**tak**	**tak**	**nie**
DeepSeek — BruteForce	**nie**	**nie**	**nie**
DeepSeek — NN	**tak**	**tak**	**nie**
GitHub Copilot	**nie**	**nie**	**nie**
Microsoft Copilot	**nie**	**nie**	**nie**
Gemini	**nie**	**nie**	**nie**
Tabnine	**tak**	**tak**	**nie**

Tabela 4.5: Poprawność działania — macierz 15×15.

graf asymetryczny	Wykonanie	Wynik poprawny	Najkrótsza ścieżka
ChatGPT	**tak**	**tak**	**tak**
Claude	**tak**	**tak**	**tak**
DeepSeek — BruteForce	**tak**	**tak**	**tak**
DeepSeek — NN	**tak**	**tak**	**tak**
GitHub Copilot	**nie**	**nie**	**nie**
Microsoft Copilot	**tak**	**nie**	**nie**
Gemini	**tak**	**tak**	**tak**
Tabnine	**tak**	**tak**	**tak**

Tabela 4.6: Poprawność działania — graf asymetryczny.

Błędy implementacyjne

Niektóre implementacje zawierają błędy logiczne, które skutkują negatywnymi wynikami testów.

1. **Zapis poza zakresem tablicy (GitHub Copilot)** W warunku końcowym wykonywany jest zapis poza rozmiarem bufora ścieżki, co skutkuje wyjątkiem podczas testu. Poniżej przedstawiono fragment kodu generujący ten wyjątek:

```
int[] path = new int[n + 1];
...
if (count == n && graph[currPos][0] > 0) {
    path[count]   = currPos;
    path[count+1] = 0;  // ArrayIndexOutOfBoundsException
    return cost + graph[currPos][0];
}
```

2. **Niepełna permutacja (MS Copilot)** Generowana trasa pomija jedno z miast, co jest wychwytywane przez walidator ścieżki. W efekcie we wszystkich testach, niezależnie od rozmiaru instancji, otrzymano niepoprawny wynik.

Podsumowanie

W tabelach poprawności widać zatem trzy wzorce:

1. Dokładne algorytmy uzyskują optymalny koszt na małych instancjach, lecz przy większych instancjach narzut pamięciowy uniemożliwia uzyskanie wyniku na wykorzystywanym sprzęcie.

2. Rozwiązania wykorzystujące heurystykę NN wytwarzają poprawny cykl, ale często nie uzyskują optymalnego wyniku; są jednak zdolne do zwrócenia trasy dla większych instancji.

3. Implementacje z błędami indeksowania lub konstrukcji trasy nie przechodzą testów nawet dla najmniejszego grafu.

4.4.1 Odporność na błędne dane wejściowe (złożone zapytanie)

Tabela 4.7 zestawia reakcję poszczególnych implementacji na wadliwe dane wejściowe. Wartość *tak* oznacza, że metoda `solveTSP` prawidłowo przerwała działanie, zgłaszając `IllegalArgumentException` (zgodnie z wymaganiami testów), natomiast *nie* oznacza brak oczekiwanej walidacji (np. cicha akceptacja danych, rzucenie innego wyjątku lub błąd wykonania).

Przypadek	ChatGPT	Claude	DeepSeek BF	DeepSeek NN	GitHub Copilot	MS Copilot	Gemini	Tabnine
macierz jako `null`	nie	tak	tak	tak	nie	nie	tak	tak
macierz 0×0	nie	tak	tak	tak	nie	nie	tak	tak
macierz niekwadratowa	nie	nie	tak	tak	nie	nie	nie	tak
macierz z wierszem równym `null`	nie	nie	nie	nie	nie	nie	nie	nie

Tabela 4.7: Odporność na błędne dane wejściowe — implementacje dla złożonego kontekstu.

Wnioski

Najlepiej wypadają warianty DeepSeek (zarówno brute force, jak i NN) oraz Tabnine — poprawnie odrzucają trzy z czterech typów błędnych danych (`null`, macierz 0×0 i niekwadratową). Claude i Gemini przechodzą dwa pierwsze przypadki, lecz nie wychwytują macierzy niekwadratowej. Najsłabiej w tej próbie wypadają ChatGPT, GitHub Copilot i Microsoft Copilot, które nie spełniły żadnego z kryteriów walidacyjnych.

Warto zwrócić uwagę na uniwersalną słabość wszystkich badanych rozwiązań: brak obsługi sytuacji, w której którykolwiek wiersz macierzy jest równy `null`. Ten przypadek powinien kończyć się jednoznacznym `IllegalArgumentException`, tymczasem każde rozwiązanie przepuszcza błąd dalej (zwykle kończąc się innym wyjątkiem w głąb algorytmu). W praktyce oznacza to, że większość generatorów dodała podstawowe strażniki wejścia (null-check rozmiaru i kształtu), ale nie weryfikuje *spójności każdej z referencji wierszy.*

Rekomendacja

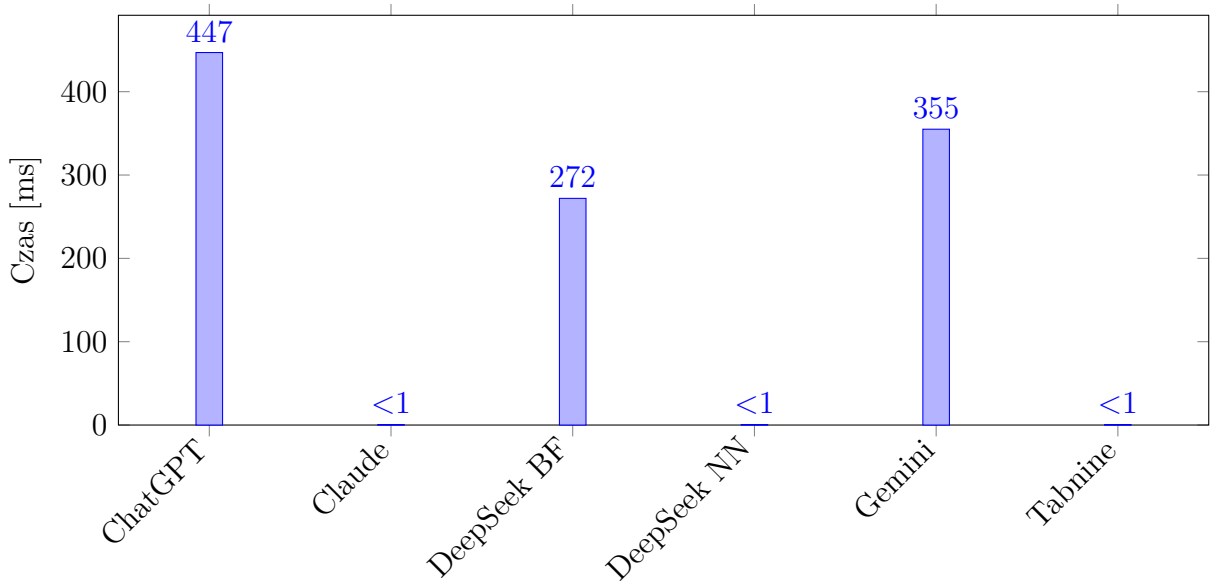
Minimalny zestaw strażników wejścia, który pozwoliłby zaliczyć wszystkie cztery przypadki, to:

- sprawdzenie, że `graph != null` i `n > 0`
- sprawdzenie, że macierz jest kwadratowa (`graph[i] != null` oraz `graph[i].length == n` dla każdego wiersza)
- w razie naruszenia — rzucenie `IllegalArgumentException` z czytelnym komunikatem.

Problemy te najprawdopodobniej można by rozwiązać poprzez podanie do narzędzi jeszcze dokładniejszego kontekstu, definiującego na jakiego typu błędne dane wejściowe oczekujemy zabezpieczeń.

4.4.2 Czas wykonania i zajętość pamięci

W tej sekcji zestawiono wydajność rozwiązań dla instancji 10×10 oraz 15×15. Na wykresach ujęto *wyłącznie* implementacje, które zwróciły poprawny wynik (niekoniecznie optymalny). Wszystkie wartości stanowią średnią z 10 niezależnych pomiarów.

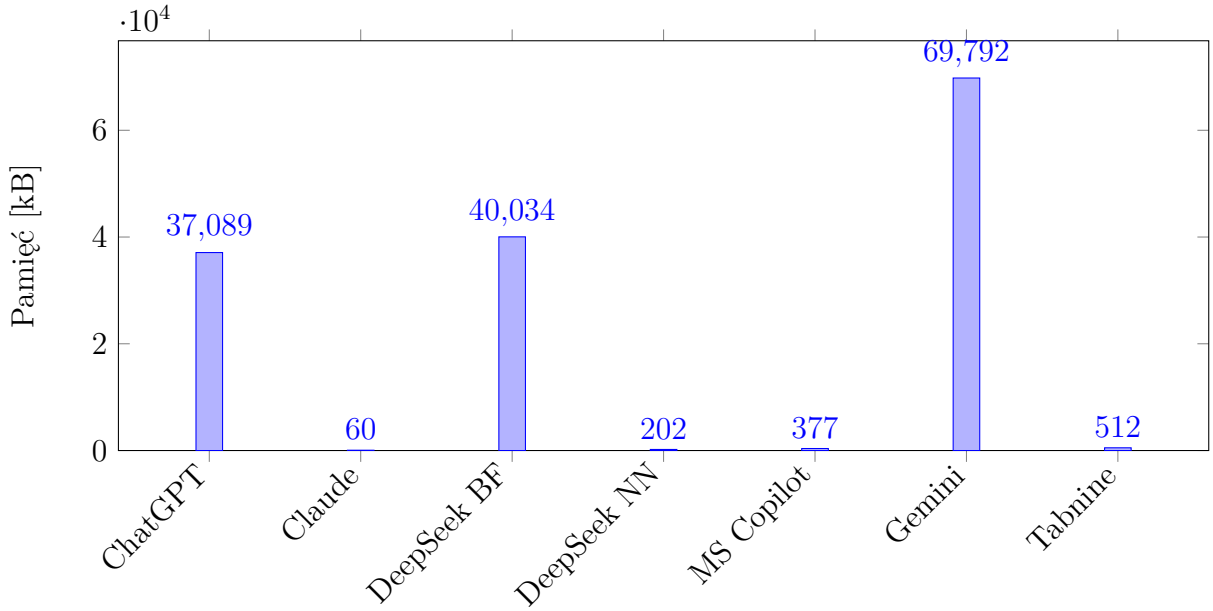


Rysunek 4.1: Czas wykonania — instancja 10×10. Ujęto tylko implementacje, które zwróciły wynik.

Omówienie wykresu czasu — 10×10 (Rys. 4.1)

Widzoczny jest wyraźny rozdział między przeglądowymi podejściami a heurystykami NN: ChatGPT (447 ms), DeepSeek BF (272 ms) i Gemini (355 ms) wymagają istotnie

więcej czasu obliczeń, co jest zgodne z wykładniczą złożonością pełnego przeglądu. Heurystyki NN (Claude, DeepSeek NN, Tabnine) uzyskują czasy bliskie zeru, co odzwierciedla niewielki narzut obliczeniowy algorytmu zachłannego. Wszystkie słupki pokazują średnią z 10 przebiegów.



Rysunek 4.2: Zajętość pamięci — instancja 10×10. Ujęto tylko implementacje, które zwróciły wynik.

Omówienie wykresu pamięci — 10×10 (Rys. 4.2)

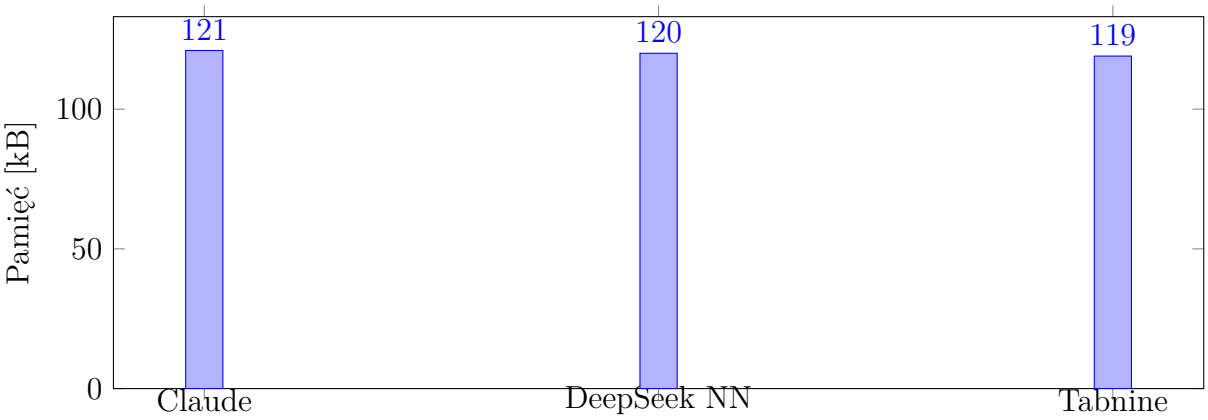
Ślad pamięciowy wzmacnia powyższy obraz: brute force zużywa więcej pamięci (ChatGPT $\sim$ 37,1 MB, DeepSeek BF $\sim$ 39,1 MB, Gemini $\sim$ 68,2 MB), podczas gdy NN zużywa o kilka rzędów mniej (Claude $\sim$ 60 kB, DeepSeek NN $\sim$ 202 kB, Tabnine $\sim$ 512 kB). Na tle pozostałych wyróżnia się Gemini z największą zajętością pamięci — wynik ten może korelować z większą złożonością kognitywną implementacji (np. utrzymywanie dodatkowych, częściowo redundantnych struktur danych czy kopiowanie ścieżek).

Omówienie pomiaru czasu — 15×15

Dla instancji 15×15 wszystkie rozwiązania, które zwróciły wynik, uzyskały zmierzony czas równy 0 ms (po zaokrągleniu do milisekund), dlatego nie przedstawiono wyników tego pomiaru w postaci wykresu.

Omówienie wykresu pamięci — 15×15 (Rys. 4.3)

Wynik uzyskały jedynie heurystyki NN; ich zajętość pamięci mieści się w przedziale od dziesiątek do kilkuset kilobajtów (Claude $\sim$ 119 kB, DeepSeek NN $\sim$ 120 kB, Tabnine $\sim$ 121 kB). Różnice między NN są niewielkie z punktu widzenia skali zadania.



Rysunek 4.3: Zajętość pamięci — instancja 15 × 15. Ujęto tylko implementacje, które zwróciły wynik (heurystyki NN).

Wnioski

Podsumowując testy wydajnościowe, uzyskano następujące wnioski:

1. Dla 10 × 10 brute force przegrywa zarówno czasowo, jak i pamięciowo z NN — różnice są rzędu kilku rzędów wielkości.

2. W rodzinie NN czasy są bliskie zeru, a ślad pamięciowy niski; obserwowane rozrzuty należy interpretować jako szum pomiarowy (średnie z 10 uruchomień).

3. Wyjątkiem w teście 10 × 10 jest szczególne użycie pamięci w przypadku rozwiązania Google Gemini, sugerujące „cięższą" implementację (bogatszą logikę sterującą i dodatkowe bufory), mimo tej samej klasy podejścia algorytmicznego.

Rozdział 5

Podsumowanie i wnioski

Celem pracy było empiryczne zbadanie tego, w jakim stopniu współczesne narzędzia AI do generowania kodu potrafią samodzielnie wytworzyć poprawne i wydajne implementacje dla złożonych problemów informatycznych - dla przykładu wykorzystano klasyczny problem komiwojażera w Javie. Zagadnienie objęło zarówno prostą sytuację promptową (ograniczony kontekst), jak i złożoną (rozszerzony kontekst), a ocena dotyczyła trzech wymiarów:

- poprawności i optymalności rozwiązań,
- wydajności obliczeniowej i pamięciowej,
- jakości kodu w świetle statycznej analizy (SonarCloud/SonarQube, Qodana, inspekcje IntelliJ IDEA).

5.1 Sposób rozwiązania problemu

Zaprojektowano zunifikowany warsztat badawczy: wspólny interfejs `TspAlgorithm`, spójny dla wszystkich rozwiązań zestaw testów i zbiór danych wejściowych (macierze 4×4, 10×10, 15×15 oraz graf asymetryczny). Każde narzędzie AI (ChatGPT, Claude, DeepSeek w wariantach brute force i NN, GitHub Copilot, Microsoft Copilot, Google Gemini, Tabnine) dostało analogiczne polecenia w wariancie prostym i złożonym. Dla wyników liczbowych wykonywano po 10 przebiegów i raportowano średnie.

Kod analizowano narzędziami statycznymi: SonarQube, Qodana oraz wbudowane inspekcje IntelliJ IDEA.

Cechy przyjętej metodyki badań:

- Jednolite testy umożliwiły porównywalność wyników (te same instancje, ten sam interfejs, te same reguły walidacji).

- Połączenie trzech źródeł analizy statycznej (SonarCloud, Qodana, IntelliJ) dało pełniejszy obraz jakości kodu.
- Zastosowanie dwóch poziomów kontekstu promptu pozwoliło wykazać istotę prompt engineeringu.
- Łatwa rozszerzalność środowiska testowego o kolejne narzędzia, przypadki testowe.

Ograniczenia:

- Ograniczona dokładność pomiarów czasu oraz zużycia pamięci.
- Tylko jeden testowany język (Java) i ograniczona liczba instancji — wyniki są reprezentatywne, lecz nie wyczerpujące.

5.2 Najważniejsze obserwacje empiryczne

Wpływ kontekstu. Przy ograniczonym kontekście zapytania rozrzut w implementacjach był wyraźnie większy: częściej pojawiały się naiwne, przeglądowe rozwiązania *brute force*, aczkolwiek wyróżnić należy tutaj rozwiązanie Claude demonstrujące różne podejścia algorytmiczne w przejrzysty sposób. Po doprecyzowaniu wymagań, interfejsu i ograniczeń danych — rozwiązania stały się bardziej jednolite względem siebie. W całym badaniu dominowały dwie klasy podejść: brute force oraz NN.

Poprawność i optymalność. Na małych instancjach (4×4, 10×10) implementacje *brute force* zwykle osiągały optimum. Dla 15×15 wynik zwracały wyłącznie heurystyki NN: poprawny cykl Hamiltona, lecz nieoptymalny koszt. To potwierdza klasyczny dylemat przy wyborze podejścia algorytmicznego: dokładność / skalowalność.

Wydajność. Dla instancji 10×10 czasy przeszukiwania pełnego były rzędu setek ms, podczas gdy NN dawały czasy poniżej milisekundy oraz niższe zużycie pamięci. Dla **Gemini** ślad pamięciowy był zauważalnie większy (co sugeruje bardziej złożone lub redundantne struktury danych). Dla 15×15 wyniki uzyskały wyłącznie NN, otrzymane one zostały w bardzo krótkim czasie i z małym narzutem pamięciowym.

Jakość kodu i odporność. Najczęstsze problemy dotyczyły utrzymywalności (zbędne importy, `System.out/err` zamiast loggera, `size()==0` zamiast `isEmpty()`, publiczne pola, powtarzane literały, zbyt złożone metody) oraz walidacji wejścia. Najpoważniejsze błędy wystąpiły w wariancie złożonego zapytania w rozwiązaniach dwóch narzędzi:

- GitHub Copilot — zapis poza zakresem bufora ścieżki,
- Microsoft Copilot — trasy pomijające jedno miasto.

W testach odporności najlepiej radziły sobie implementacje sprawdzające `null` oraz przypadki 0×0; macierz niekwadratowa była weryfikowana mniej konsekwentnie, a test „wiersz = `null`" nie zaliczały wszystkie rozwiązania.

5.3 Rekomendacje praktyczne

Wyniki badania podkreślają istotę inżynierii zapytań podczas pracy z narzędziami opartymi na sztucznej inteligencji. Na podstawie niniejszej pracy można sformułować kilka rekomendacji w tej dziedzinie.

5.3.1 Dokładne zdefiniowanie celu oraz metody rozwiązania

Jest to kluczowa kwestia w przypadku inżynierii zapytań: im precyzyjniej określone zostanie *co* i *jak* ma zostać zbudowane, tym mniejsze ryzyko halucynacji i błędów. W praktyce warto jawnie wskazać:

- klasę metody (np. „NN z losowym startem i 2-opt"),
- oczekiwane interfejsy (`TspAlgorithm.solveTSP(int[][])` $\rightarrow$ `TspResult`),

Równie ważne jest z góry zadeklarowanie kompromisu *czas / dokładność / pamięć* (np. „preferuj czas, akceptuj suboptymalność $\leq 10\%$") i deterministyczności albo — przeciwnie — wielokrotnych uruchomień i wyboru najlepszego wyniku.

5.3.2 Walidacja wejścia/wyjścia - kontrakty poprawności

Przy projektowaniu zapytania warto jawnie zdefiniować cechy danych wejściowych oraz wyjściowych. Dla opisywanego w pracy przypadku, warto w zapytaniu uwzględnić takie kwestie jak: `requireNonNull` dla macierzy i wierszy, test kwadratowości (i symetrii, jeśli wymagana), odrzucenie ujemnych wag, $n \geq 2$; po stronie wyjścia — cykl Hamiltona domknięty do miasta startowego, długość ścieżki $= n + 1$, brak powtórzeń. Jasne *kryteria akceptacji* (np. „wszystkie testy poprawności przechodzą; $\geq 95\%$ testów odporności kończy się pomyślnie") zwiększają trafność odpowiedzi w pierwszej iteracji.

5.3.3 Higiena kodu

Szczególnie istotną kwestią w przypadku wieloletnich projektów prowadzonych przez duże zespoły programistów jest higiena kodu, którą również warto uwzględnić przy projektowaniu zapytania. Dobrą praktyką jest zdefiniowanie i umieszczeniu w zapytaniu zestawu zasad, których narzędzie powinno przestrzegać podczas generowania kodu. Przykładowe zasady w tej dziedzinie to np.

- logger zamiast `System.out`,
- stałe dla powtarzanych literałów,
- `isEmpty()` zamiast `size()==0`,
- stosowanie prywatnych pól tam gdzie to możliwe,
- redukcja złożoności długich metod.

Zalecane jest tutaj wpięcie narzędzia do statycznej analizy kodu (np. SonarQube) i ustawienie bramki jakości.

Podsumowując: należy traktować LLM jako współautora, któremu trzeba dostarczyć precyzyjny kontekst, wyraźne priorytety i twarde kryteria. Taki kontrakt prowadzi do stabilniejszych, czytelniejszych i bardziej skalowalnych rozwiązań niż „otwarte" prośby bez wymagań.

5.4 Możliwości rozwoju i nowe kierunki

Praca może być dalej rozwijana w następujących kierunkach:

- **Inne języki**: porównać generacje dla Pythona, C#, C++ (wpływ idiomatyki i bibliotek).
- **Bogatsze konteksty**: szablony promptów wymuszające konkretne warianty (np. alogrytm Helda-Karpa, algorytm genetyczny) z kryteriami akceptacji.
- **Większe instancje i nowe metryki**: np. $|V| = 50 \ldots 200$, stabilność wyników (wariancja), energochłonność, zużycie CPU.

5.5 Realizacja celu pracy

Cel pracy został zrealizowany: dostarczono metodyczne porównanie narzędzi AI w generowaniu rozwiązań TSP w Javie, wykazując wpływ *prompt engineeringu*, identyfikując wzorce skuteczności i słabości, oraz formułując praktyczne rekomendacje co do sposobu użycia LLM w zadaniach algorytmicznych.

Co więcej, praca skutecznie wypełnia lukę w literaturze, która najczęściej koncentruje się na wynikach i wydajności obliczeń, a w mniejszym stopniu na samych implementacjach i jakości kodu generowanego przez modele. W odróżnieniu od większości publikowanych prac, w niniejszej pracy równolegle oceniano: czytelność i styl (nazewnictwo, modularność), utrzymywalność (złożoność kognityczna, powtarzalność, logowanie), higienę kodu, a także walidację danych wejściowych i odporność na błędy. Zaproponowana procedura

łączy kryteria funkcjonalne (poprawność, optymalność) z inżynierskimi (jakość i bezpieczeństwo implementacji), co pozwala na bardziej holistyczną ocenę generacji kodu i stanowi użyteczny punkt odniesienia dla dalszych badań i praktyki.

5.6 Podsumowanie końcowe

Praca pokazuje, że sztuczna inteligencja jako asystent programisty potrafi dostarczyć użyteczne rozwiązania, ale wymaga świadomego prowadzenia: precyzyjnych promptów, rygorystycznych testów, automatycznej analizy statycznej i jasnych kryteriów jakości. W takich warunkach narzędzia stają się realnym wsparciem. Dalsze prace powinny pójść w kierunku rozwoju o kolejne przypadki kontekstowe, umożliwiające porównanie rozwiązań opartych o konkretny algorytm.

Bibliografia

[1] Anthropic. *Claude*. 2023. URL: https://www.anthropic.com/claude (term. wiz. 31. 08. 2025).

[2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang i in. „Program synthesis with large language models". W: *arXiv preprint arXiv:2108.07732* (2021).

[3] Richard Bellman. „Dynamic Programming Treatment of the Travelling Salesman Problem". W: *Journal of the ACM* 9.1 (1962), s. 61–63.

[4] Irwan Bello, Hieu Pham, Quoc V. Le, Mohammad Norouzi i Samy Bengio. „Neural Combinatorial Optimization with Reinforcement Learning". W: *arXiv preprint arXiv:1611.09940*. 2016, s. 1–15.

[5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pinto, Jared Kaplan, Harri Edwards, Yuri Burda i in. „Evaluating Large Language Models Trained on Code". W: *arXiv preprint arXiv:2107.03374* (2021).

[6] Maurice Clerc i James Kennedy. „Particle Swarm Optimization". W: *Proceedings of the 2002 IEEE International Conference on Evolutionary Computation*. 2002, s. 1470–1477.

[7] George A. Croes. „A method for solving traveling-salesman problems". W: *Operations Research* 6.6 (1958), s. 791–812.

[8] George B. Dantzig, Ray Fulkerson i Selmer Johnson. „Solution of a Large-Scale Traveling-Salesman Problem". W: *Journal of the Operations Research Society of America* 2.4 (1954), s. 393–410.

[9] DeepSeek. *DeepSeek Coder*. 2025. URL: https://deepseek.com/ (term. wiz. 31. 08. 2025).

[10] Marco Dorigo i Luca M. Gambardella. „Ant Colony System: A Cooperative Learning Approach to the Traveling Salesman Problem". W: *IEEE Transactions on Evolutionary Computation* 1.1 (1997), s. 53–66.

[11] Daniel Fried, Aitor Lewkowycz, Eric Zelikman i in. „Emergent Algorithm Design from Large Language Models". W: *arXiv preprint arXiv:2409.19922* (2024).

[12] GitHub & OpenAI. *GitHub Copilot*. 2021. URL: https://github.com/features/copilot (term. wiz. 31. 08. 2025).

[13] David E. Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading, MA: Addison-Wesley, 1989. ISBN: 978-0201157673.

[14] Google DeepMind. *Google Gemini*. 2023. URL: https://gemini.google.com/ (term. wiz. 31. 08. 2025).

[15] Chaitanya K. Joshi, Thomas Laurent i Xavier Bresson. „Learning TSP Requires Rethinking Generalization". W: *International Conference on Learning Representations (ICLR)*. 2019, s. 1–12.

[16] Scott Kirkpatrick, C. Daniel Gelatt i Mario P. Vecchi. „Optimization by Simulated Annealing". W: *Science* 220.4598 (1983), s. 671–680.

[17] Eugene L. Lawler, Jan Karel Lenstra, Alexander H.G. Rinnooy Kan i David B. Shmoys. *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. New York: John Wiley & Sons, 1985, s. 361. ISBN: 978-0471904137.

[18] LeetCode. *LeetCode Online Judge*. https://leetcode.com/. Accessed: 2025-08-31.

[19] Raymond Li, Lewis Tunstall, Edward Beeching, Francesco Maggiore, Teven Le Scao i in. „StarCoder: May the source be with you!" W: *arXiv preprint arXiv:2305.06161* (2023).

[20] Yaqin Liu, Yue Wang i et al. „Where Are Large Language Models for Code Generation on GitHub?" W: *arXiv preprint arXiv:2406.19544* (2024). URL: https://arxiv.org/abs/2406.19544.

[21] Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008. ISBN: 978-0132350884.

[22] Microsoft. *Microsoft Copilot*. 2023. URL: https://copilot.microsoft.com/ (term. wiz. 31. 08. 2025).

[23] Erik Nijkamp, Shuyuan Tu, Ruixiang Pang, Hiroaki Hayashi, Yixuan Wang, Yujia Xu, Lijuan Mou i in. „CodeGen: An open large language model for code with multi-turn program synthesis". W: *arXiv preprint arXiv:2203.13474* (2022).

[24] OpenAI. *ChatGPT*. 2022. URL: https://chat.openai.com/ (term. wiz. 31. 08. 2025).

[25] Md Sultanul Islam Ovi, Nafisa Anjum, Tasmina Haque Bithe, Md Mahabubur Rahman i Mst Shahnaj Akter Smrity. „Benchmarking ChatGPT, Codeium, and GitHub Copilot: A Comparative Study of AI-Driven Programming and Debugging Assistants". W: *arXiv preprint arXiv:2409.19922* (2024). URL: https://arxiv.org/abs/2409.19922.

[26] Ruchika Pandey, Prabhat Singh, Raymond Wei i Shaila Shankar. „Transforming Software Development: Evaluating the Efficiency and Challenges of GitHub Copilot in Real-World Projects". W: *arXiv preprint arXiv:2406.17910* (2024). URL: https://arxiv.org/abs/2406.17910.

[27] IBM Research. *Larger context windows give language models more background to consider*. 2024. URL: https://research.ibm.com/blog/larger-context-window (term. wiz. 31. 08. 2025).

[28] D. J. Rosenkrantz, R. E. Stearns i P. M. Lewis II. „An analysis of several heuristics for the traveling salesman problem". W: *SIAM Journal on Computing* 6.3 (1977), s. 563–581.

[29] Md Kamrul Siam, Huanying Gu i Jerry Q. Cheng. „Programming with AI: Evaluating ChatGPT, Gemini, AlphaCode, and GitHub Copilot for Programmers". W: *arXiv preprint arXiv:2411.09224* (2024). URL: https://arxiv.org/abs/2411.09224.

[30] Tabnine. *Tabnine AI Assistant for Software Developers*. 2023. URL: https://www.tabnine.com/ (term. wiz. 31. 08. 2025).

[31] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser i Illia Polosukhin. „Attention Is All You Need". W: *Advances in Neural Information Processing Systems*. 2017, s. 5998–6008.

[32] Burak Yetiştiren, Işık Özsoy, Miray Ayerdem i Eray Tünzün. „Evaluating the Code Quality of AI-Assisted Code Generation Tools: An Empirical Study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT". W: *arXiv preprint arXiv:2304.10778* (2023). URL: https://arxiv.org/abs/2304.10778.

Dodatki

Spis skrótów i symboli

AI	sztuczna inteligencja (ang. *artificial intelligence*)
LLM	duży model językowy (ang. *large language model*)
TSP	problem komiwojażera (ang. *travelling salesman problem*)
ATSP	asymetryczny problem komiwojażera (ang. *asymmetric travelling salesman problem*)
RLHF	Uczenie ze wzmocnieniem z ludzką informacją zwrotną (ang. *reinforced learning from human feedback*)
API	interfejs programowania aplikacji (ang. *application programming interface*)
NN	algorytm najbliższego sąsiada (ang. *nearest neighbour*)
BF	pełny przegląd rozwiązań (ang. *brute force*)
GA	algorytm genetyczny (ang. *genetic algorithm*)
ACO	optymalizacja kolonii mrówek (ang. *ant colony optimization*)
SA	wyżarzanie symulowane (ang. *simulated annealing*)
2-opt	lokalne doszukiwanie z wymianą dwóch krawędzi (ang. *2-opt local search*)
CI/CD	ciągła integracja oraz ciągłe dostarczanie/wdrażanie (ang. *continuous integration / continuous delivery (deployment)*)
IDE	zintegrowane środowisko programistyczne (ang. *integrated development environment*)
JVM	wirtualna maszyna Javy (ang. *Java Virtual Machine*)
JUnit	framework do testów jednostkowych (ang. *JUnit*)
NPE	wyjątek pustej referencji (ang. *NullPointerException*)
NP	klasa złożoności obliczeniowej NP (ang. *nondeterministic polynomial time*)
n	liczba miast/wierzchołków

V	zbiór wierzchołków grafu
E	zbiór krawędzi grafu
G	graf, zwykle $G = (V, E)$
$d(i, j)$	koszt/dystans pomiędzy wierzchołkami i oraz j
$\|V\|$	liczba wierzchołków grafu
$\mathbb{E}$	wartość oczekiwana
$\mathcal{O}(\cdot)$	porządek złożoności obliczeniowej (notacja *big-O*)

Lista dodatkowych plików, uzupełniających tekst pracy

W systemie do pracy dołączono dodatkowe pliki zawierające:

- źródła programu (środowisko testowe),
- arkusz kalkulacyjny z wynikami testów.

Spis rysunków

3.1 Przykładowy raport wygenerowany przez narzędzie Qodana. 30
3.2 Przykładowy raport wygenerowany przez narzędzie SonarQube. 31

4.1 Czas wykonania — instancja 10 × 10. Ujęto tylko implementacje, które zwróciły wynik. 49
4.2 Zajętość pamięci — instancja 10 × 10. Ujęto tylko implementacje, które zwróciły wynik. 50
4.3 Zajętość pamięci — instancja 15 × 15. Ujęto tylko implementacje, które zwróciły wynik (heurystyki NN). 51

Spis tabel

4.1 Zestawienie uwag statycznych i wielkości kodu (ograniczony kontekst) . . . 37
4.2 Statyczna analiza klas wygenerowanych na podstawie złożonego zapytania. *Razem = SonarCloud + IDEA + Qodana.* 45
4.3 Poprawność działania — macierz 4×4. 46
4.4 Poprawność działania — macierz 10×10. 46
4.5 Poprawność działania — macierz 15×15. 47
4.6 Poprawność działania — graf asymetryczny. 47
4.7 Odporność na błędne dane wejściowe — implementacje dla złożonego kontekstu. 48